

ABSTRACT

Large language models face a trade-off between inference accuracy and computational cost. Cheap direct prompting fails on multi-step reasoning, while deliberate strategies such as Tree-of-Thought (ToT) achieve high accuracy at the cost of thousands of tokens and long latencies. This work presents CascadeRoute, a reinforcement learning router that selects, for each incoming query, one of six joint actions combining model size (3B or 7B) and reasoning strategy (Direct, Chain-of-Thought, Graph-of-Thought, Tree-of-Thought). Because queries are independent and routing decisions do not alter future states, we formulate the problem as a contextual multi-armed bandit and solve it online with Disjoint LinUCB. The context is a 390dimensional vector built from a Sentence-BERT embedding, simple query statistics, domain indicators and the remaining token budget. A multi-objective reward trades correctness against token and latency cost, and a self-consistency confidence gate escalates uncertain answers to stronger strategies. Evaluated on 3,712 questions from GSM8K, LogiQA and CommonsenseQA with local Qwen2.5 models, CascadeRoute reaches 81.4% accuracy, statistically close to the always-7B-ToT baseline (82.0%), while using 27.4% fewer tokens. It also outperforms UCB1, Thompson Sampling, ϵ -greedy, DQN, rule-based and random policies in mean reward. These results show that lightweight contextual bandits can learn difficultyaware routing policies without offline pretraining or the instability of deep RL.

Keywords: Reinforcement Learning, Contextual Bandits, LinUCB, LLM Routing, Chainof-Thought, Tree-of-Thought, Graph-of-Thought, Cost-Aware Inference, Safe RL, SelfConsistency, Sentence-B

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	Abstract	V
	List of Tables	VI
	List of Figures	VII
	List of Abbreviations	VIII
1	INTRODUCTION	1
1.1	Background and Motivation	1
1.2	Problem Statement	2
1.3	Reinforcement Learning Perspective	2
1.4	Objectives and Contributions	3
1.5	Organisation of the Report	3
1.6	Scope and Limitations	3
1.7	Summary of Key Findings	4
2	LITERATURE REVIEW	5
2.1	Cost-Aware Large Language Model Inference	5
2.2	Learned LLM Routing and Unified Model Cascading	6
2.3	Model Cascading and Query-Dependent Computational Allocation	8
2.4	Contextual Multi-Armed Bandits and LinUCB	8
2.5	Chain-of-Thought and Tree-of-Thought Reasoning	10
2.6	Graph-of-Thought Reasoning and Self-Consistency	10
2.7	Research Gap and Motivation for CascadeRoute	11
3	DATASET AND PROPOSED METHODOLOGY	13
3.1	Dataset Description and Experimental Benchmark	13
3.1.1	GSM8K Dataset	15
3.1.2	LogiQA Dataset	15
3.1.3	CommonsenseQA Dataset	15
3.2	Query Preprocessing and Contextual State Representation	16
3.2.1	Semantic Representation Using Sentence-BERT	17
3.2.2	Query-Level Features	17
3.2.3	Remaining Computational Budget	17
3.3	Proposed CascadeRoute Architecture	18
3.3.1	Model and Reasoning Action Space	19
3.4	Contextual-Bandit-Based Routing Methodology	20
3.4.1	LinUCB Action Selection	20
3.4.2	Exploration–Exploitation Mechanism	21

3.4.3	Online Parameter Update	22
3.5	Adaptive Reasoning and Confidence-Based Escalation	22
3.5.1	Multi-Temperature Consistency Check	22
3.5.2	Adaptive Escalation Strategy	24
3.5.3	Interaction Between LinUCB and the Confidence Gate	25
3.6	Complete CascadeRoute Workflow and Proposed Algorithm	25
3.6.1	Step-by-Step Operational Procedure	25
3.6.2	Proposed CascadeRoute Algorithm	27
3.6.3	Mathematical Objective of CascadeRoute	28
4	SYSTEM DESIGN AND ARCHITECTURE	29
4.1	System Architecture Overview	29
4.2	Architectural Components	30
4.2.1	Query Input and Processing Module	30
4.2.2	Contextual State Construction Module	31
4.2.3	LinUCB Routing Module	31
4.2.4	Action Selection and Strategy Mapping Module	32
4.3	LLM Inference and Reasoning Architecture	33
4.4	Confidence Gate and Escalation Architecture	34
4.5	Reward and Online Learning Architecture	35
4.6	Complete System Data Flow	36
5	METHODOLOGY	37
5.1	Experimental Methodology	37
5.2	Model and Reasoning Configuration	37
5.3	Reinforcement Learning Environment	38
5.4	LinUCB Implementation	38
5.5	Confidence Gate Implementation	39
5.6	Evaluation Procedure	39
6	RESULTS AND DISCUSSION	40
6.1	Experimental Evaluation Setup	40
6.2	Performance Against Fixed Inference Strategies	40
6.3	Analysis of the Cost–Accuracy Trade-Off	41
6.4	Analysis of Learned Routing Behaviour	42
6.5	Comparison with Alternative RL and Heuristic Methods	43
6.6	Discussion of LinUCB Performance	44
6.7	Discussion of the Confidence-Based Escalation Mechanism	45
6.8	Overall Discussion	45
7	EXPERIMENTAL RESULTS	46
7.1	Experimental Dataset	46

7.2	Evaluation Metrics	47
7.3	Results of Fixed Strategy Evaluation	47
7.4	Token Efficiency of CascadeRoute	48
7.5	Learned Routing Distribution	49
7.6	Comparison of Reinforcement Learning Algorithms	50
7.7	Reward-Based Performance Analysis	51
7.8	Confidence Gate Experimental Observation	52
7.9	Overall Experimental Findings	53
8	CONCLUSION AND FUTURE WORK	54
8.1	Conclusion	54
8.2	Key Contributions of the Proposed Work	55
8.3	Limitations of the Present Work	56
8.4	Future Work	56
8.4.1	Expansion to Larger and More Diverse LLM Ecosystems	56
8.4.2	Dynamic Budget-Aware Routing	57
8.4.3	Advanced Uncertainty Estimation	57
8.4.4	Multi-Objective Reinforcement Learning	58
8.4.5	Energy-Aware LLM Routing	58
8.4.6	Continuous Online Learning	59
8.4.7	Integration with Retrieval-Augmented Generation	59
8.4.8	Real-Time and Multi-User Deployment	60
8.4.9	Hardware and Edge Deployment	60
8.4.10	Extension from Contextual Bandits to Full Reinforcement Learning	61
8.5	Overall Future Direction	61
8.6	Final Conclusion	62
9	REFERENCES	63

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
Table 2.1	Comparative Analysis of Literature Relevant to CascadeRoute	11
Table 2.2	Literature Gap Analysis and CascadeRoute Contribution	12
Table 3.1	Experimental Dataset Distribution	14
Table 3.2	Components of the CascadeRoute Contextual State	18
Table 3.3	CascadeRoute Action Space	19
Table 3.4	Proposed CascadeRoute Algorithm	27
Table 4.1	Architectural Mapping of CascadeRoute Components	32
Table 4.2	Model and Reasoning Strategy Architecture	33
Table 6.1	Evaluation Dataset Composition	40
Table 6.2	Performance of Fixed Inference Strategies and CascadeRoute	40
Table 7.1	Experimental Dataset Distribution	46
Table 7.2	Performance of Fixed Strategies and CascadeRoute	48
Table 7.3	Learned CascadeRoute Routing Distribution	49
Table 7.4	Comparison of Reinforcement Learning and Heuristic Routing Algorithms	50

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 3.1	Distribution of the Experimental Dataset Across GSM8K, LogiQA, and CommonsenseQA	14
Figure 3.2	Construction of the 390-Dimensional Contextual State	16
Figure 3.3	Overall CascadeRoute System Architecture	19
Figure 3.4	LinUCB-Based Contextual Action Selection in CascadeRoute	20
Figure 3.5	Exploration–Exploitation Behaviour of the LinUCB Routing Agent	21
Figure 3.6	Online Parameter Update Mechanism of the LinUCB Agent	22
Figure 3.7	Progressive Reasoning Escalation Sequence in CascadeRoute	24
Figure 3.8	Complete Operational Workflow of the Proposed CascadeRoute System	27
Figure 3.9	Integrated Decision-Making Process of the CascadeRoute Methodology	28
Figure 3.10	Multi-Temperature Confidence Gate Used for Response Validation	23
Figure 3.11	Interaction Between LinUCB Routing and the Confidence Gate	25
Figure 4.1	Overall System Architecture of CascadeRoute	30
Figure 4.2	Model and Reasoning Strategy Hierarchy in CascadeRoute	34
Figure 4.3	Confidence Validation and Adaptive Escalation Architecture	35
Figure 4.4	Closed-Loop Reward and Online Learning Architecture of CascadeRoute	36
Figure 7.1	Accuracy Comparison Between Fixed Inference Strategies and CascadeRoute	48
Figure 7.2	Average Token Consumption Comparison Between Fixed Strategies and CascadeRoute	49
Figure 7.3	Learned Distribution of CascadeRoute Routing Decisions	50
Figure 7.4	Distribution of Queries Across CascadeRoute Reasoning Strategies	51

1.INTRODUCTION

1.1 Background and Motivation

Large Language Models (LLMs) have become the default tool for question answering, mathematical problem solving and logical reasoning. Their quality, however, depends heavily on how they are prompted and how much computation is spent on each query. Direct prompting returns an answer in a single step and costs only about a hundred tokens, but it often fails on problems that need multi-step reasoning. Structured strategies such as Chain-of-Thought (CoT), Graph-of-Thought (GoT) and Tree-of-Thought (ToT) improve accuracy by letting the model reason through intermediate steps or explore several solution paths. They do so at a much higher cost, with ToT consuming close to two thousand tokens per query and taking considerably longer to respond. Larger models add a second axis to this trade-off, since a 7B model is stronger than a 3B model but is also more expensive to run.

In practice, user queries vary widely in difficulty. A factual or commonsense question rarely needs deep reasoning, while a multi-step arithmetic or formal logic problem does. Applying the most powerful configuration to every query wastes resources, and applying the cheapest configuration to every query sacrifices accuracy. This creates a clear need for a mechanism that adapts the amount of computation to the difficulty of each query.

1.2 Problem Statement

The problem addressed in this work is dynamic, cost-aware LLM routing. Given an incoming query and a limited token budget, the system must decide which model size (3B or 7B) and which reasoning strategy (Direct, CoT, GoT or ToT) to use. The goal is to maximise answer accuracy while keeping token consumption and latency as low as possible. Static policies that always use one configuration cannot do this, because they sit at a single point on the cost-accuracy curve. Hand-written rule-based policies are brittle, because keyword rules cannot capture the true difficulty of a question. The routing decision must therefore be learned from feedback.

1.3 Reinforcement Learning Perspective

Routing maps naturally onto the reinforcement learning framework of an agent observing a state, taking an action and receiving a reward. In CascadeRoute, the state is a 390-dimensional vector composed of a Sentence-BERT embedding of the query, simple length features, domain indicators and the percentage of token budget remaining. The action space consists of six discrete model- strategy combinations. The reward combines a positive term for a correct answer with penalties for token usage and latency, so the agent is encouraged to save computation without being rewarded for wrong answers.

A key observation is that the routing environment is stateless. The choice made for one query does not change the next query that arrives, so each decision is a single-step problem and not part of a sequential Markov Decision Process. This is the setting of a contextual multi-armed bandit. We therefore adopt Disjoint LinUCB, which maintains a

ridge-regression model for each action and balances exploitation of known good actions with exploration of uncertain ones through an upper confidence bound. LinUCB learns online from every query, needs no offline pre-training, and has established regret guarantees under linear reward assumptions. Deep reinforcement learning methods such as DQN were evaluated as a comparison, because they are designed for sequential decision problems and add training complexity that this setting does not require.

To keep exploration safe, CascadeRoute also includes a confidence gate. The selected strategy is run at two sampling temperatures (0.2 and 0.8), and if the two answers disagree, the query is escalated to a stronger strategy. This protects against costly mistakes while the agent is still learning.

1.4 Objectives and Contributions

The objectives of this project are to formulate LLM routing as a reinforcement learning problem, to design a state, action and reward structure suited to it, and to compare several learning algorithms on a common benchmark. The main contributions are as follows:

- (i) A contextual bandit formulation of joint model-size and reasoning-strategy selection, with a state that includes a dynamic budget signal so the agent can become more frugal as the budget declines.
- (ii) An online LinUCB router combined with a self-consistency confidence gate that escalates uncertain answers to stronger strategies.
- (iii) A comparative evaluation of LinUCB against UCB1, Thompson Sampling, epsilon-greedy, DQN, rule-based and random policies on 3,712 questions from GSM8K, LogiQA and CommonsenseQA, using locally hosted Qwen2.5 models.
- (iv) Empirical evidence that the learned router reaches 81.4% accuracy, close to the 82.0% of the always-7B-ToT baseline, while using 27.4% fewer tokens, together with an analysis of the routing behaviour the agent learned for each domain.

1.5 Organisation of the Report

The remainder of this report is organised as follows. Section 2 describes the RL environment, including the state, action space and reward function. Section 3 presents the algorithms, with emphasis on LinUCB and the confidence gate, and the baselines used for comparison. Section 4 describes the datasets, models and experimental setup. Section 5 reports and discusses the results, including the algorithm comparison and the learned routing policy. Section 6 concludes the report and outlines directions for future work.

1.6 Scope and Limitations

This work focuses on routing between two locally hosted model sizes (Qwen2.5 3B and 7B) and four reasoning strategies (Direct, CoT, GoT and ToT), giving a discrete action space of six joint actions. Evaluation is restricted to three public benchmarks covering mathematical reasoning (GSM8K), logical reasoning (LogiQA) and commonsense reasoning (CommonsenseQA). Queries are treated as independent of one another, which is the assumption that justifies the contextual bandit formulation. Cost is measured in generated tokens and wall-clock latency, not in monetary terms or energy use.

Several limitations should be kept in mind when interpreting the results. First, the routing policy is learned and evaluated on the same stream of benchmark questions, so the reported accuracy includes the exploration phase and is not a measurement on a fully held-out test set. Second, the LinUCB regret guarantees assume that expected reward is approximately linear in the context features, an assumption that a sentence embedding only satisfies approximately. Third, the results come from two small open-source models and three English-language benchmarks, and they may not carry over to larger models, proprietary APIs or open-ended generation tasks. Finally, the confidence gate relies on agreement between two sampled answers, which detects inconsistency but cannot detect an answer that is consistently wrong.

1.7 Summary of Key Findings

The experiments show that the contextual bandit router achieves 81.4% accuracy over the 3,712 evaluation queries, compared with 82.0% for the always-7B-ToT baseline, while reducing the average token consumption by 27.4%. The static baselines confirm the cost-accuracy trade-off that motivates the work: the cheapest configuration (3B Direct) reaches 53.9% accuracy at about 107 tokens per query, whereas 7B ToT reaches 82.0% at about 1,959 tokens. The static 7B GoT configuration reaches the highest single-strategy accuracy at 83.5% with fewer tokens than ToT, so the savings reported above are measured against ToT and should be read in that context.

The learned policy also shows difficulty-aware behaviour. Nearly half of the queries (47.8%) were routed to cheap direct answers, about a third (34.2%) to Chain-of-Thought, and only 17.7% to the expensive GoT or ToT strategies, which were used mainly for complex mathematical and logical problems. When compared with the other learning methods, LinUCB obtained the best mean reward and the best token efficiency, and it learned online without offline pre-training and at lower computational cost than DQN. These findings support the view that a lightweight contextual bandit is a suitable and efficient tool for cost-aware LLM routing.

2. LITERATURE REVIEW

The rapid development of Large Language Models (LLMs) has significantly improved the performance of artificial intelligence systems across natural language understanding, question answering, mathematical reasoning, logical inference, and commonsense reasoning. However, improvements in model capability are generally accompanied by increased computational requirements, inference latency, memory consumption, and token usage. Consequently, deploying the most powerful model and the most computationally intensive reasoning mechanism for every input is neither computationally efficient nor economically sustainable. A simple factual query may be answered adequately by a relatively small model using direct inference, whereas a complex mathematical or logical problem may require a more capable model together with an explicit reasoning strategy. This variation in query difficulty has motivated research into adaptive model selection, model cascading, LLM routing, contextual decision-making, and inference-time reasoning. The proposed CascadeRoute framework lies at the intersection of these research directions. Rather than treating model selection and reasoning strategy as independent problems, CascadeRoute considers the joint selection of a language model and an inference strategy. The routing problem is formulated as a contextual decision-making problem in which characteristics of the incoming query and available computational budget provide contextual information, while different combinations of model capacity and reasoning strategy form the possible actions. The selected route is evaluated using a reward that considers answer correctness together with computational factors such as token consumption and latency. This formulation is motivated by previous work on cost-aware LLM inference, learned model routing, model cascading, contextual multi-armed bandits, Chain-of-Thought (CoT), Tree-ofThoughts (ToT), Graph-of-Thoughts (GoT), and self-consistency.

2.1 Cost-Aware Large Language Model Inference

The increasing diversity of available language models creates an important optimization problem: selecting an appropriate model for a query without unnecessarily sacrificing either answer quality or computational resources. Chen, Zaharia, and Zou introduced FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance [1], one of the important early studies addressing this problem from the perspective of economical LLM utilization. The authors observed that available LLM services exhibit substantial heterogeneity in both capability and inference cost. Consequently, using a single powerful model for every query may lead to unnecessary expenditure when less expensive models are capable of producing sufficiently accurate answers. FrugalGPT [1] considers three broad strategies for reducing the inference cost of LLM applications: prompt adaptation, LLM approximation, and LLM cascading. Among these, LLM cascading is particularly relevant to adaptive routing systems. Instead of relying exclusively on one model, a cascade can employ multiple models and determine whether the result produced at an earlier, cheaper stage is sufficient or whether processing should continue through a more capable model. This changes the inference problem from selecting a universally optimal model into selecting an appropriate amount of computation for each individual input.

The experimental findings presented by Chen et al. [1] demonstrate the potential of such adaptive inference. Their FrugalGPT implementation learns combinations of LLMs that can be used for different queries. The study reports that, in its evaluated settings, FrugalGPT can match the performance of the strongest individual LLM while reducing cost by up to 98%, or improve accuracy over the strongest individual model while operating under the same cost constraint. These findings establish an important principle: the highest-cost model need not be used for every input in order to achieve high aggregate performance. This principle directly motivates CascadeRoute. However, the proposed work extends the optimization objective beyond choosing among LLMs. In CascadeRoute, computational expenditure also depends on the reasoning strategy employed by a model. A direct response, Chain-of-Thought reasoning, Tree-of-Thought exploration, and Graph-of-Thought reasoning can have significantly different computational requirements even when executed using the same underlying model. Thus, while FrugalGPT primarily establishes that intelligent model combinations can reduce LLM inference cost, CascadeRoute generalizes this idea toward the joint selection of model capacity and reasoning depth. FrugalGPT therefore provides the first major conceptual foundation for the present study: LLM inference should be adaptive to individual queries, and efficiency should be optimized together with predictive performance rather than independently of it.

2.2 Learned LLM Routing and Unified Model Cascading

Building on the general idea of cost-aware inference, subsequent research has investigated whether the routing decision itself can be learned from data. Ong et al. proposed RouteLLM: Learning to Route LLMs with Preference Data [2], which directly addresses the problem of dynamically routing queries between language models of different capabilities. RouteLLM is particularly relevant to CascadeRoute because both systems reject the assumption that every incoming request should be processed by the same LLM. RouteLLM [2] considers the common practical setting in which a stronger model provides higher response quality but incurs greater cost, whereas a weaker model is cheaper but may fail on difficult queries. Instead of statically assigning all inputs to either model, RouteLLM learns a routing function that estimates when the stronger model is necessary. The framework makes use of preference data to learn the relative suitability of stronger and weaker models for different prompts. The significance of this formulation is that routing becomes query dependent. The correct decision is no longer simply to minimize model cost or maximize raw model accuracy. Rather, the router must estimate whether the expected improvement from the stronger model justifies its additional inference cost for the current input. Ong et al. [2] demonstrate that learned routing can substantially reduce inference expenditure while maintaining response quality. Their evaluation reports cost reductions exceeding two times without sacrificing response quality in the studied settings. The work also investigates router generalization, including routing between models that were not necessarily present in the same form during router training. The relevance of this work to CascadeRoute is substantial. The proposed system contains both a routing decision and an escalation mechanism. The contextual-bandit policy initially chooses a route based on the query representation and available budget. A confidence mechanism can subsequently determine whether the obtained response should be accepted or whether computation should be escalated toward a stronger reasoning route. In this sense, CascadeRoute incorporates characteristics of both routing and cascading. Taken together, RouteLLM [2] and the unified cascade-routing framework [3] demonstrate a clear evolution in LLM inference research. RouteLLM shows that model selection can be learned from data, while Dekoninck et al. establish that routing and cascading need not be treated as mutually exclusive mechanisms. CascadeRoute builds upon these observations by applying adaptive decision-making not merely to a set of LLMs but to a structured set of model $\times$ reasoning-strategy configurations, while incorporating computational budget into the decision context.

2.3 Model Cascading and Query-Dependent Computational Allocation

The idea that inputs of different difficulty should receive different amounts of computation predates recent LLM routing frameworks. Varshney and Baral, in *Model Cascading: Towards Jointly Improving Efficiency and Accuracy of NLP Systems* [4], investigate whether every input instance truly requires inference through the largest available model. Their central observation is that many instances are sufficiently simple to be solved by lower-capacity models, while only a subset requires larger and more computationally demanding models. The model-cascading framework presented in [4] employs collections of models with varying capacities. Rather than executing the largest model indiscriminately, the cascade attempts to exploit smaller models whenever their predictions are sufficient and reserves larger models for more difficult instances. This formulation is highly relevant to resource-aware NLP because it recognizes that computational requirements are inherently instance dependent. Varshney and Baral [4] report substantial computational savings in their experiments. In one of the reported settings involving three models, model cascading saved up to 88.93% of computation while also producing accuracy improvements of up to 2.18%. Their experiments further indicate that adding suitable models to a cascade can increase opportunities for computational efficiency. For example, increasing model size is only one mechanism for increasing inference capability. Additional reasoning procedures can also increase the amount of computation assigned to a query. A smaller model with explicit reasoning may outperform direct inference on some queries, while a larger model using direct inference may be sufficient for others. More complex problems may benefit from structured search mechanisms such as ToT or GoT. Consequently, a purely model-capacity-based cascade may not exploit the complete range of inference-time choices available in modern LLM systems. The model-cascading literature [4], together with FrugalGPT [1], RouteLLM [2], and unified cascade routing [3], therefore establishes the first major research direction underlying CascadeRoute: adaptive computational allocation across heterogeneous inference configurations. However, choosing among these configurations requires a learning mechanism capable of considering the characteristics of each incoming query while balancing exploration, exploitation, quality, and cost. This requirement motivates the contextual-bandit component discussed in the following subsection.

2.4 Contextual Multi-Armed Bandits and LinUCB

Adaptive routing can naturally be viewed as a sequential decision problem. At each interaction, the system observes information about the current input, chooses one among several possible actions, and receives feedback regarding the quality of that choice. This structure closely resembles the contextual multi-armed bandit framework. A foundational application of contextual bandits was presented by Li, Chu, Langford, and Schapire in A Contextual-Bandit Approach to Personalized News Article Recommendation [5]. Although the original application concerns personalized news recommendation rather than LLM inference, the underlying decision structure is highly transferable. In their formulation, the learning algorithm observes contextual information associated with users and candidate articles, selects an article, receives user-click feedback, and continuously adapts its article-selection policy. The important distinction between contextual and context-free bandits is that the expected value of an action depends on the observed context. A context-free strategy may learn that one action performs well on average, but it cannot determine whether another action is preferable for a particular type of input. Li et al. [5] demonstrate the practical importance of contextual information using a large Yahoo! dataset containing more than 33 million events. Their reported results show a 12.5% click lift compared with a standard context-free bandit approach, with the contextual advantage becoming greater under data scarcity. This concept maps naturally to LLM routing. In CascadeRoute, an action cannot be judged as universally optimal. A computationally inexpensive direct route may be ideal for a straightforward commonsense query but inadequate for a difficult multi-step mathematical problem. Conversely, an expensive Tree-of-Thought route may provide strong reasoning capability but represent unnecessary computation for a simple factual question. Therefore, the expected reward of each route depends strongly on the context of the current query. The theoretical foundation for the linear contextual-bandit approach is further developed by Chu, Li, Reyzin, and Schapire in Contextual Bandits with Linear Payoff Functions [6]. The authors study contextual bandits under the assumption that the expected payoff of an action can be represented as a linear function of contextual features. They analyze an efficient upperconfidence-bound algorithm and establish a high-probability regret bound of order $\sqrt{KT}$. The Upper Confidence Bound principle provides a systematic solution to the exploration–exploitation dilemma. For each action, the learner considers both its estimated expected reward and an uncertainty term. Actions with strong historical performance receive high exploitation value, while actions for which the model remains uncertain receive an exploration bonus. As more observations are collected, uncertainty decreases and the learner progressively concentrates on actions that perform well for particular contexts. This property is central to the design philosophy of CascadeRoute. A router that always selects the currently estimated best action may converge prematurely toward a suboptimal strategy because alternative routes are never sufficiently explored. Conversely, random exploration wastes computation by repeatedly selecting expensive or unsuitable strategies without considering accumulated knowledge. LinUCB offers a principled compromise by making exploration context dependent. In CascadeRoute, the contextual representation contains semantic information about the incoming query together with explicit query characteristics and the remaining

computational budget. The available actions correspond to combinations of model capacity and reasoning strategy. After execution, the selected action receives a reward influenced primarily by correctness and reduced by computational costs such as token usage and latency. This can conceptually be expressed as The contextual-bandit literature [5], [6] therefore supplies both the conceptual and theoretical basis for treating adaptive LLM routing as an online learning problem. Li et al. [5] establish the practical value of context-sensitive action selection, while Chu et al. [6] provide theoretical support for efficient linear UCB methods. CascadeRoute transfers these principles from recommendation and generic bandit optimization to resource-aware LLM inference.

2.5 Chain-of-Thought and Tree-of-Thought Reasoning

Model selection alone does not determine the computational requirements of an LLM inference request. The reasoning procedure used during inference can also have a major influence on both answer quality and computational expenditure. This motivates the second dimension of the CascadeRoute action space: reasoning strategy selection. Wei et al. introduced Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [7], a foundational study demonstrating that LLM performance on complex reasoning problems can improve when the model generates intermediate reasoning steps before producing the final answer. Instead of mapping an input directly to an output, Chain-of-Thought prompting encourages the model to decompose the problem into a sequence of intermediate steps. Wei et al. [7] evaluate CoT prompting across arithmetic, commonsense, and symbolic reasoning tasks and demonstrate substantial improvements for sufficiently capable language models. A particularly notable result is obtained on GSM8K, where a 540-billion-parameter model prompted with a small number of CoT demonstrations achieved state-of-the-art performance at the time of publication. The study established that inference-time reasoning structure itself can materially affect model performance. This observation is fundamental to CascadeRoute. If reasoning strategy affects accuracy, then selecting an LLM alone is insufficient to characterize an inference route. A model operating in direct-answer mode and the same model operating with Chain-of-Thought prompting represent different computational actions. The latter may provide higher accuracy on complex reasoning tasks but generally requires additional generated tokens and therefore greater inference cost. Consequently, CoT introduces an important trade-off. Applying it universally may increase computational consumption on questions that can already be answered directly. Avoiding it entirely may reduce performance on problems requiring multi-step reasoning. CascadeRoute addresses this trade-off by allowing the routing policy to determine when the expected improvement from CoT justifies its additional computational cost. The reasoning paradigm was extended further by Yao et al. in Tree of Thoughts: Deliberate Problem Solving with Large Language Models [8]. The authors identify a limitation of conventional autoregressive inference and linear Chain-of-Thought reasoning: once the model commits to a particular reasoning path, it has limited ability to systematically explore alternatives, reconsider earlier choices, or perform strategic lookahead. Tree of Thoughts addresses this limitation by treating intermediate reasoning steps as coherent units called “thoughts” and organizing them into a search structure. Instead of generating one reasoning trajectory, the model can explore multiple candidate paths, evaluate intermediate states, perform lookahead, and backtrack when necessary [8]. This produces a form of deliberate inference that more closely resembles search-based problem solving.

2.6 Graph-of-Thought Reasoning and Self-Consistency

Besta et al. further generalized structured LLM reasoning in Graph of Thoughts: Solving Elaborate Problems with Large Language Models [9]. Whereas Chain-of-Thought represents reasoning as a sequence and Tree-of-Thoughts organizes alternatives as a tree, Graph-of-Thoughts represents generated information as an arbitrary graph. Individual thoughts correspond to vertices, while edges represent dependencies and relationships between those thoughts. This representation enables operations that are difficult to express in purely linear or tree-based structures. Multiple reasoning branches can be combined, intermediate information can be aggregated, and previously generated thoughts can be refined through graph transformations and feedback mechanisms [9]. Consequently, GoT provides a more general abstraction for orchestrating complex LLM reasoning. Besta et al. [9] demonstrate that this flexibility can improve both task quality and computational efficiency relative to alternative structured-reasoning strategies in certain tasks. For example, the paper reports a 62% improvement in sorting quality over ToT while simultaneously reducing cost by more than 31% in that evaluated setting. This result is particularly relevant to CascadeRoute because it illustrates that reasoning strategies cannot always be arranged according to a simple assumption that “more computation necessarily means better performance.” Their effectiveness depends on the structure of the problem and on how computation is organized. GoT therefore forms an important action within the CascadeRoute reasoning space. For queries requiring synthesis of multiple intermediate ideas, iterative refinement, or non-linear dependency structures, graph-based reasoning may offer advantages. For simpler queries, however, constructing and processing such a reasoning graph may introduce unnecessary overhead. The routing policy must consequently learn the contexts in which GoT is likely to justify its additional computational requirements. Another closely related issue is determining whether an obtained answer is sufficiently reliable or whether additional computation is necessary. Wang et al. address this problem through SelfConsistency Improves Chain of Thought Reasoning in Language Models [10]. Instead of relying on the single reasoning trajectory produced through greedy decoding, self-consistency samples multiple diverse reasoning paths and selects the answer that is most consistent across them. This introduces an important safety mechanism into cost-aware routing. Pure cost minimization could encourage a policy to overuse cheap routes even when their answers are unreliable. Conversely, always selecting expensive reasoning strategies would eliminate much of the efficiency benefit. A confidence gate provides an intermediate mechanism: attempt an efficient route when appropriate, assess the stability of the resulting answer, and escalate computation when confidence is insufficient. The combination of GoT [9] and self-consistency [10] therefore expands the adaptive inference problem beyond model selection. GoT demonstrates that the structure of reasoning can itself be optimized, while self-consistency demonstrates that multiple reasoning trajectories can provide information about response reliability. CascadeRoute incorporates these principles into a unified resource-aware routing framework.

• Table 2.1. Comparative Analysis of Literature Relevant to CascadeRoute

Ref.	Study	Primary Contribution	Decision/Reasoning Mechanism	Efficiency Consideration	Relevance to CascadeRoute
[1]	Chen et al., FrugalGPT	Cost-aware use of heterogeneous LLMs through prompt adaptation, approximation and cascading	Learned LLM cascade	Explicit inference-cost optimization	Establishes cost-quality optimization and adaptive LLM cascading
[2]	Ong et al., RouteLLM	Learned routing between stronger and weaker LLMs	Preference-data-based router	Balances model capability against inference cost	Closely related foundation for query-dependent LLM routing
[3]	Dekoninck et al.	Unified theoretical framework for routing and cascading	Cascade routing	Explicit cost-performance analysis	Supports combining initial routing with subsequent escalation
[4]	Varshney and Baral	Model cascading across different model capacities	Confidence/instance-dependent cascading	Reduces computation by avoiding large models for easy inputs	Supports query-dependent computational allocation
[5]	Li et al.	Contextual-bandit framework and LinUCB application	Context-aware UCB action selection	Efficient online learning	Foundation for contextual routing decisions
[6]	Chu et al.	Theoretical analysis of linear contextual bandits	Linear UCB with confidence bounds	Regret-efficient sequential learning	Mathematical basis for LinUCB routing
[7]	Wei et al.	Chain-of-Thought prompting	Sequential intermediate reasoning	Increased inference for improved reasoning	Provides a medium-complexity reasoning action
[8]	Yao et al.	Tree-of-Thoughts	Multi-path search, evaluation and backtracking	Higher inference-time computation	Provides deliberate high-complexity reasoning
[9]	Besta et al.	Graph-of-Thoughts	Graph-structured thought generation and aggregation	Considers quality-cost trade-offs	Provides non-linear structured reasoning
[10]	Wang et al.	Self-consistency decoding	Sampling and agreement across reasoning paths	Additional computation used to improve reliability	Motivates confidence checking and escalation

2.7 Research Gap and Motivation for CascadeRoute

The literature reviewed above demonstrates substantial progress in efficient LLM inference, model routing, contextual decision-making, and inference-time reasoning. Nevertheless, these research directions have largely evolved as related but distinct problems. This separation creates an important opportunity for a unified framework such as CascadeRoute. FrugalGPT [1] demonstrates that intelligent LLM cascading can substantially reduce inference cost while preserving strong performance. RouteLLM [2] shows that model routing can be learned from preference information instead of relying exclusively on manually designed thresholds. Dekoninck et al. [3] provide a theoretical framework unifying routing and cascading, while Varshney and Baral [4] establish that inputs of different difficulty should not necessarily receive inference from the same model capacity. Collectively, these studies establish the importance of adaptive model-level computation. However, these approaches primarily focus on which model or sequence of models should process an input. Modern LLM inference introduces an additional decision dimension: how the selected model should reason. Chain-of-Thought [7], Tree-of-Thoughts [8], and Graphof-Thoughts [9] demonstrate that reasoning procedures can substantially alter the problemsolving capabilities of language models. At the same time, these methods incur different amounts and structures of inference-time computation. Therefore, selecting a model without considering its reasoning strategy does not fully optimize the cost–quality trade-off. The contextual-bandit literature [5], [6] provides an appropriate mechanism for addressing this expanded decision problem. Instead of defining one globally optimal inference strategy, contextual bandits learn which action is appropriate given the current context. Furthermore, the UCB principle provides a computationally efficient mechanism for balancing exploration of uncertain actions with exploitation of configurations that have historically produced high rewards. A second gap concerns dynamic computational budget awareness. Existing routing systems commonly optimize average monetary or computational cost, but a deployed system may operate under a finite inference budget that changes over time. The optimal action when substantial budget remains may differ from the optimal action near budget exhaustion. Treating remaining budget as part of the contextual state enables the routing policy to learn such adaptive behavior. A third gap concerns joint optimization of correctness, token consumption, and latency. Routing decisions based solely on expected accuracy may overuse expensive reasoning strategies, while decisions based solely on cost may sacrifice response quality. A practical inference router should therefore optimize a multi-objective reward in which correctness remains the dominant objective while resource consumption introduces controlled penalties. A fourth gap concerns confidence-aware escalation. Adaptive routing introduces a risk that an initially inexpensive route may be selected for a difficult query. Self-consistency research [10] demonstrates that agreement among multiple reasoning trajectories contains useful information about response reliability. Combining routing with a confidence mechanism allows a system to retain the efficiency advantages of inexpensive initial actions where $(E(q_t))$ represents the semantic embedding of query (q_t) , $(F(q_t))$ represents additional query characteristics, and (B_t) represents the remaining computational budget. The contextual-bandit policy then estimates the expected utility of each action and balances

exploitation against exploration. Unlike a static cascade, the system therefore does not assume that one fixed escalation sequence is universally optimal. Instead, the initial route is selected according to contextual information. A simple query may directly receive a low-cost route, whereas a difficult query can immediately be assigned a stronger reasoning configuration when historical evidence indicates that cheaper routes are unlikely to succeed. Confidence-based escalation subsequently provides an additional mechanism for recovering from uncertain initial decisions.

Table 2.2. Literature Gap Analysis and CascadeRoute Contribution

Research Dimension	Existing Literature	Identified Limitation / Gap	CascadeRoute Direction
Cost-aware LLM inference	FrugalGPT [1]	Primarily optimizes selection/cascading among LLM services; reasoning strategy is not the central routing dimension	Jointly considers model capacity and reasoning strategy
Learned LLM routing	RouteLLM [2]	Primarily routes between stronger and weaker models	Expands actions to multiple model-reasoning combinations
Routing and cascading	Dekoninck et al. [3]	Focuses on model-selection paradigms and quality estimation	Combines learned initial routing with confidence-based escalation across reasoning configurations
Model cascading	Varshney and Baral [4]	Primarily varies model capacity	Allocates computation through both model capacity and inference-time reasoning depth
Context-aware decision-making	Li et al. [5]	Developed for contextual recommendation rather than LLM inference	Applies contextual-bandit learning to LLM inference routing
Linear contextual bandits	Chu et al. [6]	Provides generic theoretical framework rather than an LLM-specific routing architecture	Uses LinUCB-style contextual action selection for model-reasoning routes

The literature consequently suggests that the central problem is no longer simply determining whether a small or large language model is superior. Instead, the relevant question is how much inference-time computation should be allocated to each individual query and in what form that computation should be applied. Model capacity, reasoning depth, token expenditure, latency, confidence, and remaining budget all influence this decision. CascadeRoute addresses this problem by bringing together four previously related but comparatively separated research directions: cost-aware model selection, contextual-bandit learning, adaptive reasoning, and confidence-based escalation. FrugalGPT [1], RouteLLM [2], the unified routing and cascading framework [3], and model cascading [4] motivate adaptive computational allocation. The contextual-bandit

works of Li et al. [5] and Chu et al. [6] provide the learning framework required for context-sensitive online decisions. CoT [7], ToT [8], and GoT [9] establish a hierarchy of alternative reasoning mechanisms, while selfconsistency [10] motivates the use of response agreement as evidence for deciding whether further computation may be required. Accordingly, the research gap addressed by CascadeRoute can be summarized as the absence, among the reviewed works, of a unified framework that simultaneously performs queryaware, budget-aware, online selection of both LLM capacity and reasoning strategy while explicitly optimizing correctness against token and latency costs and retaining a confidence-based escalation mechanism. The proposed framework is designed to investigate whether such joint routing can approach the accuracy of computationally intensive reasoning policies while significantly reducing unnecessary inference expenditure.

3. DATASET PROVENANCE AND METHODOLOGY

3.1 Dataset Description And Experimental Benchmark

The effectiveness of an adaptive LLM routing system depends on the diversity and complexity of the queries used for evaluation. Since CascadeRoute is designed to determine how much computational effort should be allocated to an individual query, the experimental dataset must contain questions with different reasoning requirements. For this purpose, CascadeRoute is evaluated using three established reasoning benchmarks: GSM8K, LogiQA, and CommonsenseQA. The three datasets cover distinct categories of reasoning and therefore provide a heterogeneous evaluation environment. The experimental collection contains a total of 3,712 questions. GSM8K contributes 1,319 questions, LogiQA contributes 1,172 questions, and CommonsenseQA contributes 1,221 questions. The overall distribution is shown below.

Dataset	Reasoning Domain	Number of Questions
GSM8K	Mathematical reasoning	1,319
LogiQA	Logical reasoning	1,172
CommonsenseQA	Commonsense reasoning	1,221
Total	Combined evaluation set	3,712

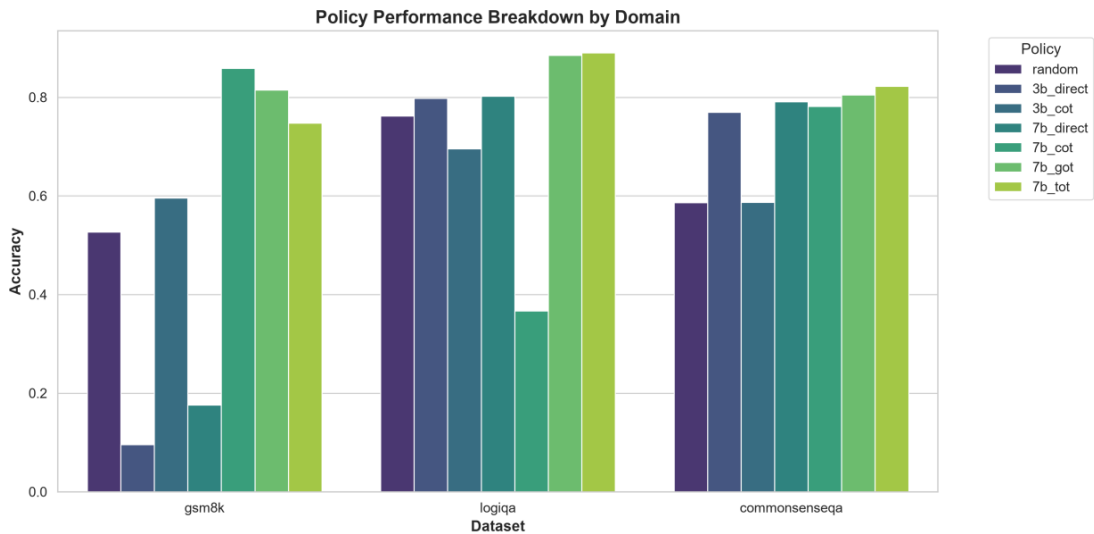


Figure 3.1 – Distribution of the Experimental Dataset Across GSM8K, LogiQA, and CommonsenseQA

The combination of these datasets is important because CascadeRoute does not aim to optimize a single type of question. Instead, it is intended to learn whether the computational requirements of a query depend on its semantic content, reasoning category, and complexity.

3.1.1 GSM8K Dataset

GSM8K, or Grade School Math 8K, is a benchmark designed to evaluate mathematical reasoning capabilities in language models. The questions are expressed as natural-language mathematical word problems and generally require multiple reasoning steps before arriving at the final numerical answer. A total of 1,319 GSM8K questions are included in the experimental collection used for CascadeRoute.

GSM8K is particularly relevant to the proposed system because mathematical problems can vary significantly in complexity. Some questions may be solved using direct inference or relatively short reasoning chains, whereas others require several intermediate calculations. This makes the dataset useful for investigating whether the routing system can allocate additional computational resources when deeper reasoning is likely to improve answer quality. For example, a simple arithmetic problem may be efficiently handled by a smaller model using direct inference, while a problem requiring multiple dependent calculations may benefit from Chain-of-Thought or a more computationally intensive reasoning strategy. Thus, GSM8K provides an important test of the relationship between query difficulty, reasoning depth, and computational cost.

3.1.2 LogiQA Dataset

LogiQA is a benchmark focused on logical reasoning. The questions require the model to interpret relationships between multiple statements, conditions, and constraints and subsequently identify the logically appropriate answer. The experimental collection contains 1,172 LogiQA questions. LogiQA is relevant to CascadeRoute because logical reasoning often requires the integration of multiple pieces of information. Some questions can potentially be solved through direct inference, while more complicated questions may require explicit step-by-step reasoning or exploration of alternative reasoning paths. Consequently, LogiQA allows the proposed routing system to evaluate whether it can identify queries for which additional reasoning computation is beneficial.

3.1.3 CommonsenseQA Dataset

CommonsenseQA is a multiple-choice benchmark designed to evaluate commonsense reasoning. The questions generally require the model to understand everyday situations, relationships, and plausible interpretations. The experimental collection contains 1,221 CommonsenseQA questions. CommonsenseQA is useful for evaluating the ability of CascadeRoute to avoid unnecessary computation. Many commonsense questions may be answered correctly using relatively inexpensive inference. Therefore, a successful routing policy should ideally recognize such cases and avoid automatically assigning expensive reasoning mechanisms. At the same time, some commonsense questions can involve ambiguity or multiple interacting concepts. Such cases may benefit from additional reasoning. Therefore, CommonsenseQA provides a useful environment for evaluating adaptive allocation of computation. The three datasets collectively provide the following coverage:

- GSM8K → mathematical reasoning • LogiQA → logical reasoning • CommonsenseQA → commonsense reasoning

This diversity allows CascadeRoute to learn and evaluate routing behavior across different types of natural-language reasoning problems.

3.2 Query Preprocessing And Contextual State Representation

The first stage of CascadeRoute is the transformation of a natural-language query into a numerical contextual state. Since the routing decision depends on the characteristics of the incoming query, the system must provide the routing policy with sufficient information about the query. The state representation used by CascadeRoute contains 390 features. These features consist of:

1. A 384-dimensional semantic embedding.
 2. Word count.
 3. Sentence count.
 4. Three category indicators.
 5. Remaining computational budget.
- The contextual state can therefore be represented as: $x_t = [E(q_t), L(q_t), S(q_t), C(q_t), B_t]$ where: x_t = contextual state at time t $E(q_t)$ = semantic embedding of query q_t $L(q_t)$ = word count of query q_t $S(q_t)$ = sentence count of query q_t $C(q_t)$ = category indicators B_t = remaining computational budget This representation allows the router to consider both the content of the query and the current computational condition of the system.

Figure 3.2 – Construction of the 390-Dimensional Contextual State

3.2.1 Semantic Representation Using Sentence-BERT

The semantic component of the state is generated using Sentence-BERT, specifically the allMiniLM-L6-v2 model. The resulting embedding contains 384 numerical dimensions. $E(q_t) \in \mathbb{R}^{384}$ Therefore:

The semantic embedding represents the meaning of the query in a continuous vector space. This enables semantically similar queries to have similar representations even when they use different words or sentence structures. For example, two mathematical questions may use different vocabulary but have similar underlying reasoning requirements. Their embeddings can capture this similarity and allow the routing policy to transfer information from previously observed decisions. The semantic representation is therefore more informative than relying exclusively on manually defined keywords.

3.2.2 Query-Level Features

In addition to the semantic representation, CascadeRoute uses explicit characteristics of the query. The current state includes:

- Word count • Sentence count • Query category indicators Word count provides a basic measure of query length, while sentence count provides an indication of the structural complexity of the input. The system also includes three category indicators corresponding to:
- Mathematics • Logic • Commonsense These indicators allow the contextual-bandit policy to differentiate between broad categories of reasoning problems.

3.2.3 Remaining Computational Budget

The final component of the contextual state is the remaining computational budget. $B_t \in [0,1]$ The budget is represented as a continuous value between 0 and 1:

where: $B_t = 1$ indicates that the available budget is essentially unused. $B_t = 0$ indicates that the available budget has been exhausted. Including budget as part of the contextual state is a key feature of CascadeRoute. Without this information, the routing policy would learn a relatively fixed relationship between query characteristics and inference actions.

With budget information, the same type of query can potentially receive different routes depending on the resources currently available. For example, when sufficient budget remains, the system may select a computationally expensive reasoning strategy for a difficult query. When the budget is low, the policy may prefer a cheaper alternative. The complete state representation is summarized in Table 3.2.

Table 3.2: Components of the CascadeRoute Contextual State

Component	Number of Features	Purpose
Sentence-BERT embedding	384	Represents semantic meaning
Word count	1	Represents query length
Sentence count	1	Represents structural complexity
Category indicators	3	Represents reasoning domain
Remaining budget	1	Represents current computational availability
Total	390	Complete contextual state

3.3 Proposed Cascaderoute Architecture

CascadeRoute is implemented as an adaptive inference pipeline in which an incoming query passes through several interconnected components. The overall processing sequence is: Query $\rightarrow$ Context Construction $\rightarrow$ LinUCB Router $\rightarrow$ Model/Reasoning Selection $\rightarrow$ LLM Execution $\rightarrow$ Confidence Evaluation $\rightarrow$ Reward Calculation $\rightarrow$ Policy Update The architecture separates the routing decision from the actual execution of the selected LLM configuration. The routing component determines what should be executed, while the inference component performs the selected computation.

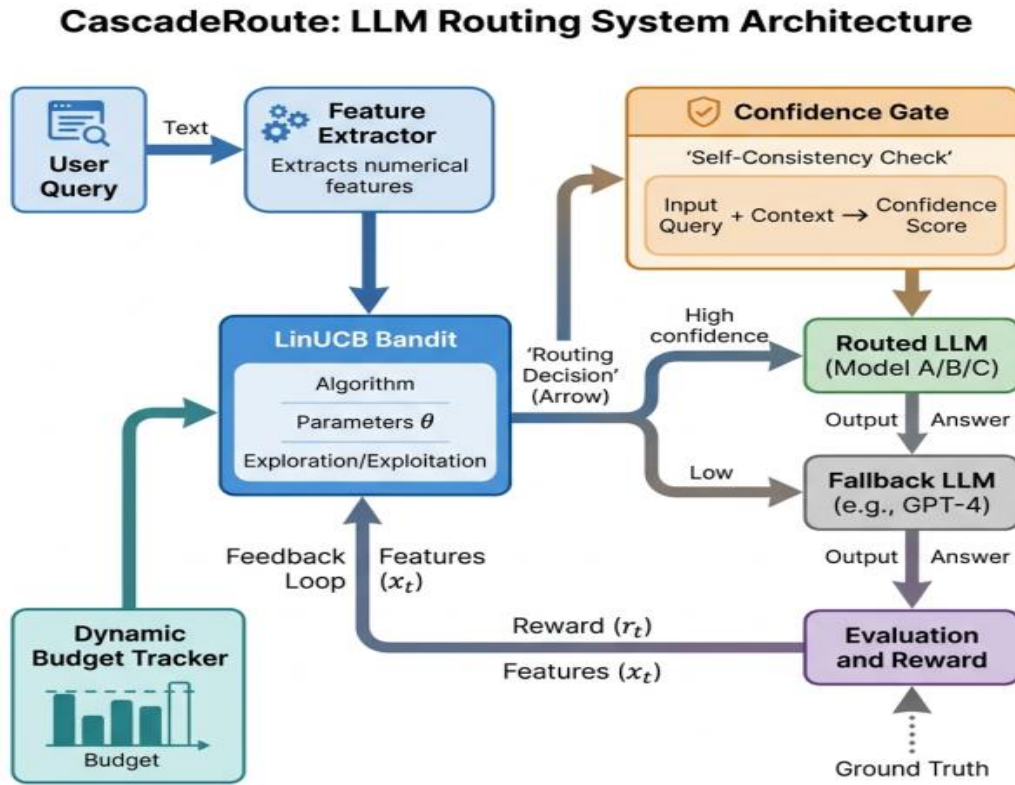


Figure 3.3 – Overall CascadeRoute System Architecture

3.3.1 Model and Reasoning Action Space

The most important feature distinguishing CascadeRoute from a conventional model router is its joint action space. Instead of selecting only between a small model and a large model, the system selects a combination of:

1. Model capacity 2. Reasoning strategy

The six primary actions used in the system are:

Table 3.3: CascadeRoute Action Space

Action	Model	Reasoning Strategy	Relative Cost
3b direct _	3B	Direct answer	Lowest
3b cot _	3B	Chain-of-Thought	Low
7b direct _	7B	Direct answer	Low
7b cot _	7B	Chain-of-Thought	Medium
7b got _	7B	Graph-of-Thought	High
7b tot _	7B	Tree-of-Thought	Highest

3.4 Contextual-bandit-based Routing Methodology

The core decision-making mechanism of CascadeRoute is formulated as a contextual multiarmed bandit problem. For every incoming query, the system observes a contextual state representing the semantic characteristics of the query, its structural properties, its category, and the remaining computational budget. Based on this state, the routing agent selects one of six available model–reasoning combinations.

The proposed methodology employs the Disjoint Linear Upper Confidence Bound (LinUCB) algorithm. LinUCB is particularly suitable for CascadeRoute because each query can be treated as an independent decision episode, while the observed context provides information about which model and reasoning strategy is likely to produce the best cost–accuracy trade-off. The algorithm combines exploitation of previously learned reward estimates with exploration of actions for which the agent has insufficient information.

3.4.1 LinUCB Action Selection

For an incoming query at time step t , the system first constructs the contextual state vector x_t . The state contains the 384-dimensional Sentence-BERT semantic embedding, question-level characteristics, category indicators, and the remaining token budget. Thus, the resulting state belongs to a 390-dimensional feature space. $p_{t,a} = \theta_a^T x_t + \alpha \sqrt{x_t^T A_a^{-1} x_t}$ where $p_{t,a}$ represents the upper confidence bound score of action a , θ_a is the learned parameter vector corresponding to action a , A_a is the action-specific covariance matrix, x_t is the contextual state vector, and α is the exploration coefficient. The first term represents exploitation because it estimates the expected reward of selecting action a for the current query. The second term represents exploration by providing an uncertainty bonus. Actions that have not been sufficiently explored for a particular type of query receive a larger uncertainty value and are therefore more likely to be selected. $a_t = \arg \max_a p_{t,a}$. The action having the maximum upper confidence bound is selected as the routing decision. The action space consists of six strategies: 3B Direct, 3B Chain-of-Thought, 7B Direct, 7B Chain-of-Thought, 7B Graph-of-Thought, and 7B Tree-of-Thought. These actions represent different levels of computational reasoning capability.

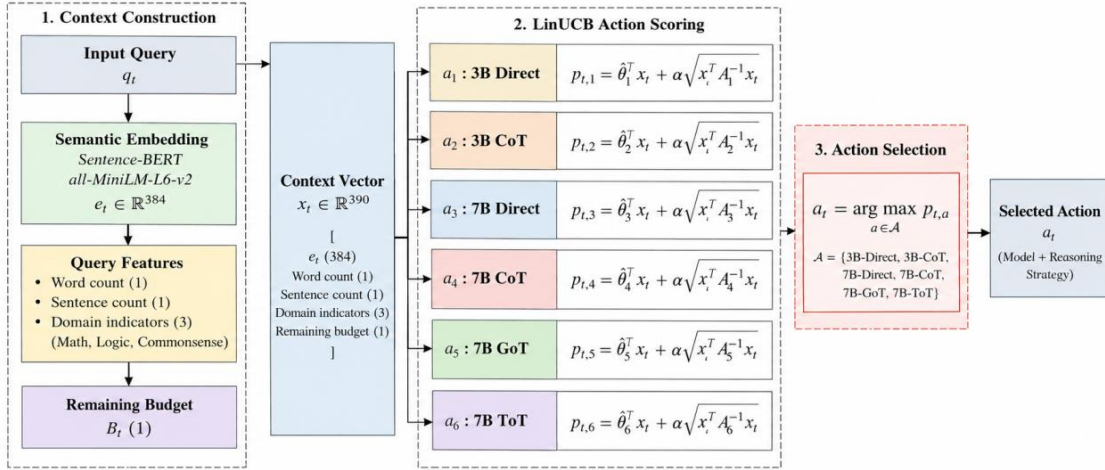


Figure 3.10: LinUCB-based contextual action selection in CascadeRoute

Figure 3.4: LinUCB-based contextual action selection in CascadeRoute

3.4.2 Exploration–Exploitation Mechanism

A fundamental requirement of the routing system is to balance exploration and exploitation. Exploitation allows the agent to select strategies that have historically produced high rewards for similar queries, whereas exploration allows the agent to investigate alternative strategies whose performance is uncertain. In CascadeRoute, the uncertainty term in the LinUCB score naturally provides the exploration mechanism. If a particular action has been selected only a limited number of times for a specific contextual region, its uncertainty remains relatively high. Consequently, the action can receive a larger upper confidence bound and obtain opportunities for exploration. As more queries are processed, the agent updates the parameters associated with each action. The uncertainty associated with repeatedly selected actions gradually decreases, allowing the learned policy to focus increasingly on strategies that provide high rewards for particular classes of queries. This mechanism is especially useful for LLM routing because the optimal inference strategy is query-dependent. Simple questions can often be answered using inexpensive direct inference, whereas complex mathematical or logical questions may benefit from more computationally intensive reasoning strategies.

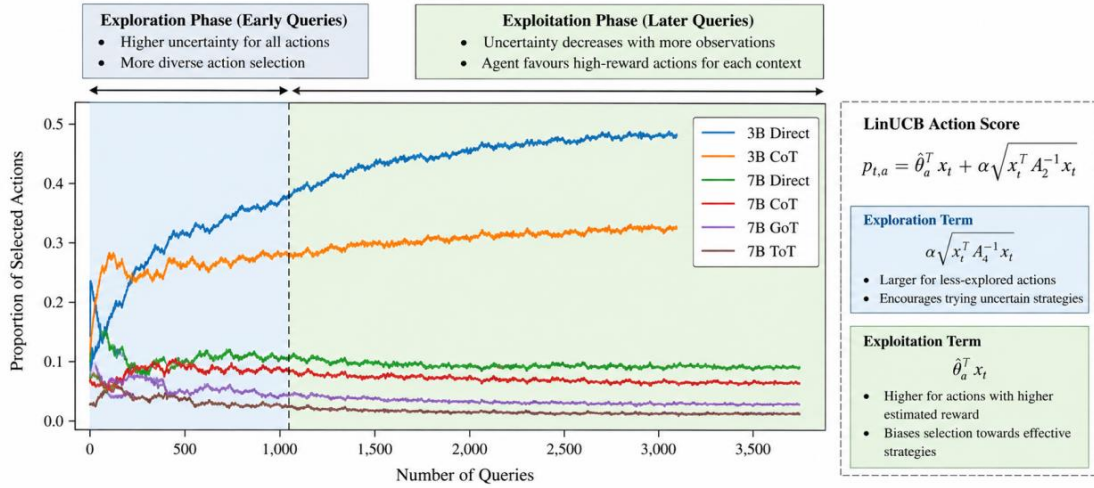


Figure 3.11: Exploration–exploitation behaviour of the LinUCB routing agent

Figure 3.5: Exploration–exploitation behaviour of the LinUCB routing agent

3.4.3 Online Parameter Update

After an action is executed, the system evaluates the resulting response and computes the reward using correctness, token consumption, and execution time. The obtained reward is then used to update the parameters associated with the selected action. For the selected action a_t , the covariance matrix is updated as follows: $A_{a,t} \leftarrow A_{a,t} + x_t x_t^T$. The corresponding reward-weighted feature vector is updated using:

$b_{a,t} \leftarrow b_{a,t} + R_t x_t$. The parameter estimate for the selected action is subsequently obtained from: $\hat{\theta}_{a,t} = A_{a,t}^{-1} b_{a,t}$. These updates enable CascadeRoute to learn continuously from its interaction with incoming queries. Unlike approaches that require a separate offline training stage, the contextual-bandit formulation allows the routing policy to adapt as new query–action–reward observations become available.

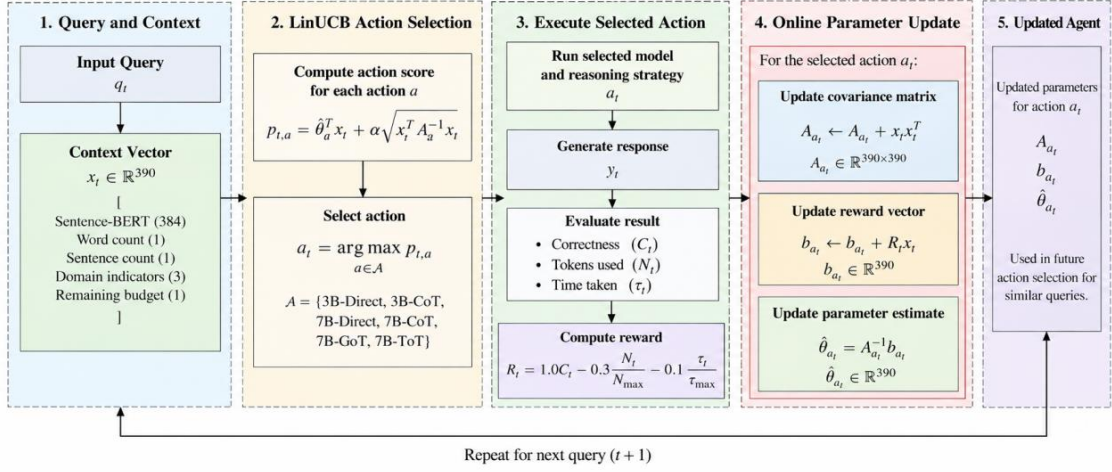


Figure 3.12: Online parameter update mechanism of the LinUCB agent

Figure 3.6: Online parameter update mechanism of the LinUCB agent

3.5 Adaptive Reasoning And Confidence-based Escalation

Although contextual bandit routing can learn an efficient model-selection policy, exploration can occasionally result in an insufficient reasoning strategy being selected for a difficult query. To address this limitation, CascadeRoute incorporates a confidence-based safety mechanism called the Confidence Gate. The Confidence Gate performs a multi-temperature consistency check. The selected strategy is executed at two different temperature settings. If the resulting answers are sufficiently consistent, the response is accepted. If the outputs conflict, the system assumes that the selected strategy may not have provided a sufficiently reliable answer and escalates the query to the next stronger reasoning strategy.

3.5.1 Multi-Temperature Consistency Check

For every selected action, CascadeRoute performs two inference passes using different temperature values. The first pass uses a low temperature to encourage focused and deterministic behaviour, while the second uses a higher temperature to introduce greater variation in the generated response. $T_1 = 0.2$ $T_2 = 0.8$ The two responses generated for the selected action are represented as: $y_{t,1} = f(q_t, a_t, T_1)$ $y_{t,2} = f(q_t, a_t, T_2)$ The responses are then compared to determine whether they exhibit sufficient agreement. The confidence decision can be represented as: $C_t = \{ 1, \text{ if } y_{t,1} \approx y_{t,2} \quad 0, \text{ if } y_{t,1} \neq y_{t,2} \}$ When the two responses agree, the system considers the selected strategy sufficiently reliable and returns the generated answer. When the responses disagree, the query is considered uncertain and is passed to the escalation mechanism.

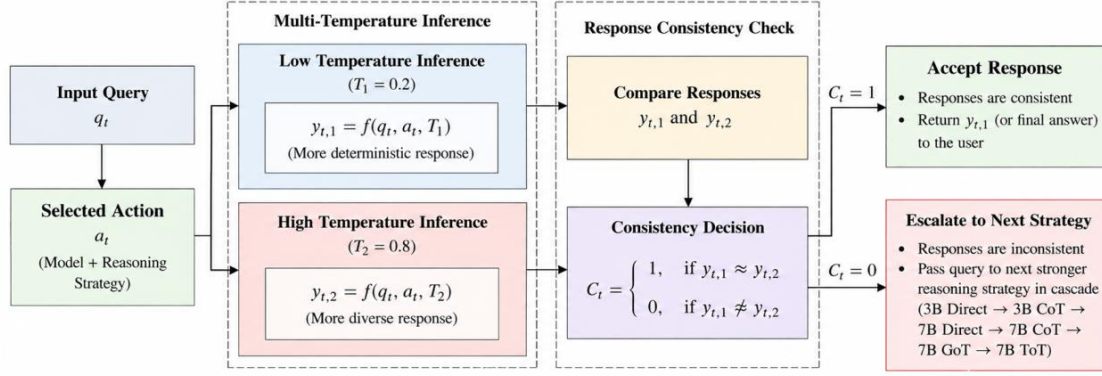


Figure 3.13: Multi-temperature confidence gate used for response validation

Figure 3.13: Multi-temperature confidence gate used for response validation

3.5.2 Adaptive Escalation Strategy

When a conflict is detected by the Confidence Gate, CascadeRoute does not immediately execute the most computationally expensive reasoning strategy. Instead, the query is progressively escalated through an ordered sequence of increasingly powerful strategies. The escalation sequence implemented in the system is: 3B Direct $\rightarrow$ 3B CoT $\rightarrow$ 7B Direct $\rightarrow$ 7B CoT $\rightarrow$ 7B GoT $\rightarrow$ 7B ToT

This ordered cascade enables the system to increase computational effort only when required. For example, a query initially routed to 3B Direct is first checked using the Confidence Gate. If the two generated responses conflict, the query is escalated to 3B Chain-of-Thought. If uncertainty persists, the system continues through the remaining strategies until a sufficiently reliable response is obtained. This approach prevents the system from unnecessarily using expensive Tree-of-Thought reasoning for queries that can be reliably answered using simpler strategies. The project overview reports that the confidence mechanism was able to catch and correct approximately 15–20% of exploratory routing mistakes during the early learning process. RL_OVERVIEW

Figure 3.7: Progressive reasoning escalation sequence in CascadeRoute

3.5.3 Interaction Between LinUCB and the Confidence Gate

The LinUCB agent and the Confidence Gate perform complementary functions. LinUCB determines the most promising strategy based on the current contextual state, whereas the Confidence Gate verifies whether the selected strategy produces a sufficiently consistent response. $a_t = \arg \max_a \square A p_{t,a}$ The overall decision process can therefore be represented as:

Decision_t $\square$ {Accept, Escalate} The selected action is then passed through the confidence validation stage:

An “Accept” decision terminates the inference process and returns the generated response. An “Escalate” decision transfers the query to the next stronger strategy in the predefined cascade. Consequently, the system combines learned routing with a deterministic safety mechanism. This combination is important because the contextual bandit is designed to explore and learn, while the confidence gate prevents uncertain exploratory decisions from being directly returned to the user.

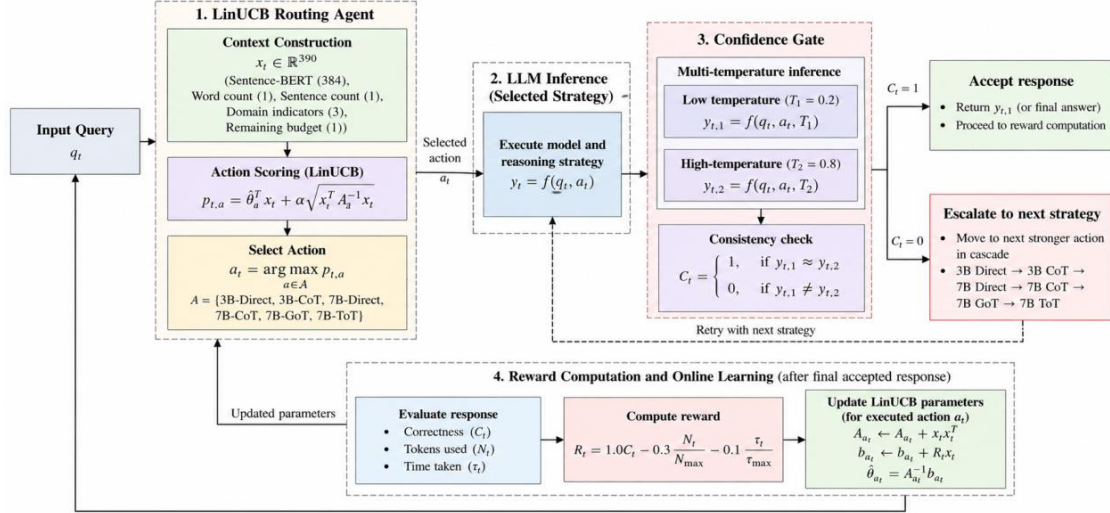


Figure 3.15: Interaction between LinUCB routing and the Confidence Gate

Figure 3.15: Interaction between LinUCB routing and the Confidence Gate

3.6 Complete Cascadroute Workflow And Proposed Algorithm

The complete CascadeRoute methodology integrates contextual feature extraction, contextualbandit-based routing, LLM inference, confidence validation, adaptive escalation, reward computation, and online learning into a unified pipeline. Each incoming query passes through these stages sequentially, allowing the system to dynamically determine the appropriate model and reasoning depth. The workflow is designed to optimise the trade-off between answer correctness and computational cost. Rather than applying the same inference strategy to every query, CascadeRoute allocates computational resources according to the characteristics and difficulty of each individual query.

3.6.1 Step-by-Step Operational Procedure

The first stage of the workflow converts the incoming query into a semantic representation using Sentence-BERT. The resulting embedding captures the semantic characteristics of the query. $E_t = \text{SBERT}(q_t)$. The query is then analysed to extract structural and categorical characteristics: $F_t = [L(q_t), S(q_t), C(q_t)]$ where $L(q_t)$ represents the query word count, $S(q_t)$ represents the sentence count, and $C(q_t)$ represents the category indicators for Math, Logic, and Commonsense. The remaining computational budget is then incorporated into the state representation. The complete contextual state is given by:

$$x_t = [E_t^T, F_t^T, B_t] x_t \in \mathbb{R}^{390} \text{ The resulting state vector has 390 dimensions:}$$

The 390-dimensional state consists of the 384-dimensional Sentence-BERT embedding, word count, sentence count, three category indicators, and the remaining budget. **RL_OVERVIEW** Once the state has been constructed, the LinUCB agent calculates the confidence-bound score for each available action and selects the action with the highest score. $a_t = \arg \max_a \square A [\theta_a^T x_t + \alpha \sqrt{(x_t^T A_a^{-1} x_t)}]$ The selected model and reasoning strategy are then executed to generate a response. The confidence gate performs two inference passes using different temperature settings: $y_{t,1} = f(q_t, a_t, T = 0.2)$ $y_{t,2} = f(q_t, a_t, T = 0.8)$ If the outputs agree, the response is accepted. If they conflict, the query is escalated to the next available strategy in the cascade. After the final response is obtained, the system calculates the reward based on correctness, token consumption, and execution time: $R_t = 1.0C_t - 0.3(N_t/N_{\max}) - 0.1(\tau_t/\tau_{\max})$ where C_t denotes correctness, N_t represents the number of tokens used, $N_{\max}$ is the maximum token budget, τ_t is the time taken, and $\tau_{\max}$ is the maximum permitted execution time. The reward formulation is the one specified for the CascadeRoute system. **RL_OVERVIEW** The calculated reward is used to update the LinUCB parameters for the selected action. Finally, the remaining computational budget is updated according to the resources consumed during inference. $B_{t+1} = B_t - \Delta B_t$

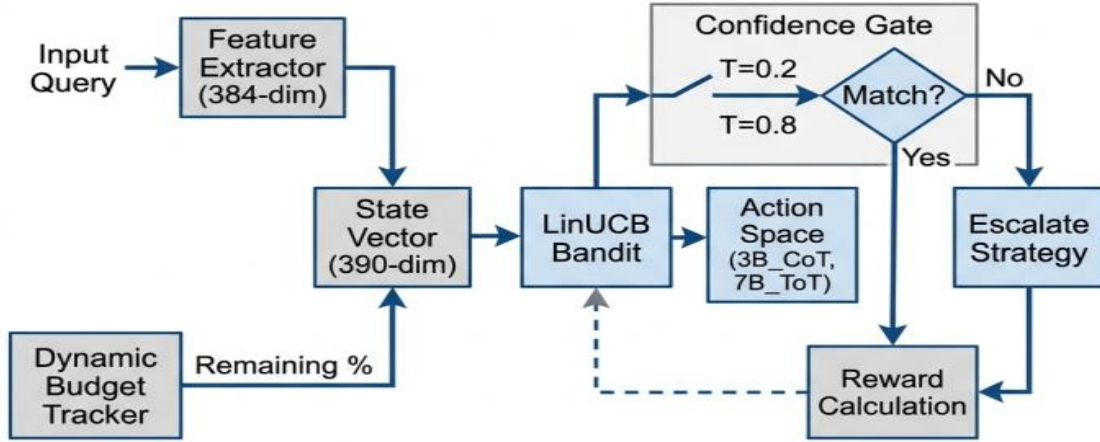


Figure 3.8: Complete operational workflow of the proposed CascadeRoute system

3.6.2 Proposed CascadeRoute Algorithm

The complete routing process is summarised in Table 3.4. The algorithm begins by constructing the contextual state, evaluating all available actions using LinUCB, and selecting the action with the highest upper confidence bound. The selected strategy is then validated using the Confidence Gate. If inconsistency is detected, the query is escalated until a reliable response is obtained. Finally, the reward is calculated and the corresponding LinUCB parameters are updated.

Table 3.4 Proposed CascadeRoute Algorithm

Step Procedure Input Incoming query q_t , remaining budget B_t , and action set A . *State Construction* Generate the Sentence-BERT embedding and extract word count, sentence count, category indicators, and remaining budget to construct $x_t \in \mathbb{R}^{390}$. *Action Scoring* Calculate the LinUCB score for every available action using the expected reward and uncertainty terms. *Action Selection* Select the action having the maximum LinUCB score. *Inference* Execute the selected model and reasoning strategy to generate a response. *Confidence Check* Execute the selected strategy at $T = 0.2$ and $T = 0.8$ and compare the resulting responses. *Escalation* If the responses conflict, move to the next stronger reasoning strategy in the predefined cascade. *Reward* Calculate the reward using correctness, token consumption, and execution time. *Parameter Update* Update A_a , b_a , and θ_a for the selected action. *Budget Update* Update the remaining computational budget according to the resources consumed. *Output* Return the validated response to the user.

Step	Procedure
Input	Incoming query q_t , remaining budget B_t , and action set A .
State Construction	Generate the Sentence-BERT embedding and extract word count, sentence count, category indicators, and remaining budget to construct $x_t \in \mathbb{R}^{390}$.
Action Scoring	Calculate the LinUCB score for every available action using the expected reward and uncertainty terms.
Action Selection	Select the action having the maximum LinUCB score.
Inference	Execute the selected model and reasoning strategy to generate a response.
Confidence Check	Execute the selected strategy at $T = 0.2$ and $T = 0.8$ and compare the resulting responses.
Escalation	If the responses conflict, move to the next stronger reasoning strategy in the predefined cascade.
Reward	Calculate the reward using correctness, token consumption, and execution time.
Parameter Update	Update A_a , b_a , and θ_a for the selected action.
Budget Update	Update the remaining computational budget according to the resources consumed.
Output	Return the validated response to the user.

3.6.3 Mathematical Objective of CascadeRoute

The overall objective of CascadeRoute is to maximise answer quality while minimising the computational resources required to obtain that answer. The routing policy therefore seeks a balance between correctness, token consumption, and inference latency. The optimal policy can be represented mathematically as: $\pi = \arg \max_{\pi} E[Q - \lambda C C - \lambda L L]^*$ where π^* represents the optimal routing policy, Q represents the quality or correctness of the generated response, C represents computational or token cost, L represents inference latency, and λC and λL represent the corresponding cost weights. In the implemented CascadeRoute system, this general objective is operationalised through the reward function: $R_t = 1.0C_t - 0.3(N_t/N_{\max}) - 0.1(\tau_t/\tau_{\max})$. The reward formulation gives correctness the highest importance while introducing penalties for excessive token consumption and inference time. This encourages the learned routing policy to select inexpensive strategies whenever they are sufficient, while still allowing computationally intensive reasoning strategies to be selected when required for difficult queries. The resulting methodology therefore combines adaptive model selection, contextual reinforcement learning, confidence-based validation, and progressive reasoning escalation within a single inference framework.

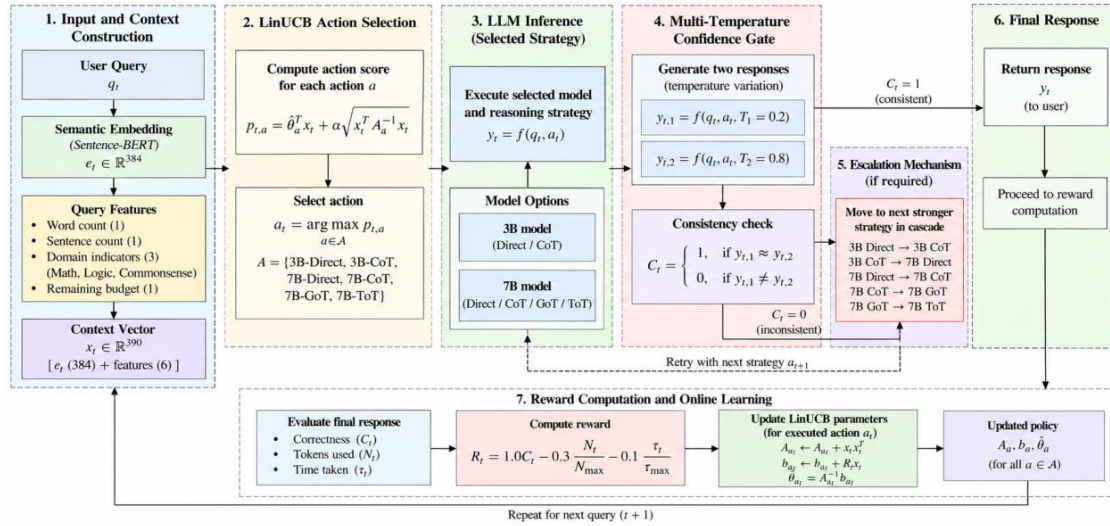


Figure 3.9: Integrated decision-making process of the CascadeRoute methodology

Ta

4. SYSTEM DESIGN AND ARCHITECTURE

The proposed CascadeRoute system is designed as an intelligent inference-routing architecture that dynamically determines the appropriate Large Language Model (LLM) and reasoning strategy for every incoming query. The architecture integrates contextual feature extraction, reinforcement learning-based decision-making, local LLM execution, confidence-based validation, adaptive escalation, reward computation, and online policy updating into a unified pipeline. The system follows the fundamental reinforcement learning interaction of state observation, action selection, environment execution, reward generation, and policy update. In CascadeRoute, the user query acts as the primary input, while the contextual state is constructed from the semantic representation of the query, its structural characteristics, its category, and the remaining computational budget. The LinUCB-based routing agent then selects one of six available inference strategies.

RL_OVERVIEW The architecture is designed around two major objectives: first, to maintain high answer accuracy by allocating stronger reasoning strategies to difficult queries; and second, to reduce unnecessary computational expenditure by using inexpensive strategies whenever they are sufficient. This design enables CascadeRoute to learn a query-dependent inference policy rather than applying a fixed model or reasoning strategy to every input.

4.1 System Architecture Overview

The overall architecture of CascadeRoute consists of five major functional stages: Query Processing, Contextual State Construction, RL-Based Routing, LLM Execution and Confidence Validation, and Reward-Based Online Learning. The first stage receives the user's natural-language query. The query is processed to obtain its semantic representation using Sentence-BERT. Additional characteristics such as word count, sentence count, and problem category are extracted and combined with the remaining computational budget. The resulting information forms the contextual state supplied to the LinUCB routing agent. The agent evaluates the available actions using their expected reward and uncertainty estimates and selects the most appropriate model–reasoning combination. The selected action is then executed through the local LLM inference environment. CascadeRoute uses locally executed Ollama-based models, with Qwen2.5 models forming part of the execution environment. The system therefore avoids requiring every routing decision to be sent to a remote inference service.

RL_OVERVIEW After inference, the Confidence Gate evaluates the consistency of the generated response using two different temperature settings. If the responses are consistent, the answer is returned. If a conflict is detected, the query is escalated to a stronger reasoning strategy. Finally, the system calculates the reward based on correctness, token consumption, and inference time. This reward is used to update the corresponding LinUCB parameters, allowing the routing policy to improve through subsequent interactions.

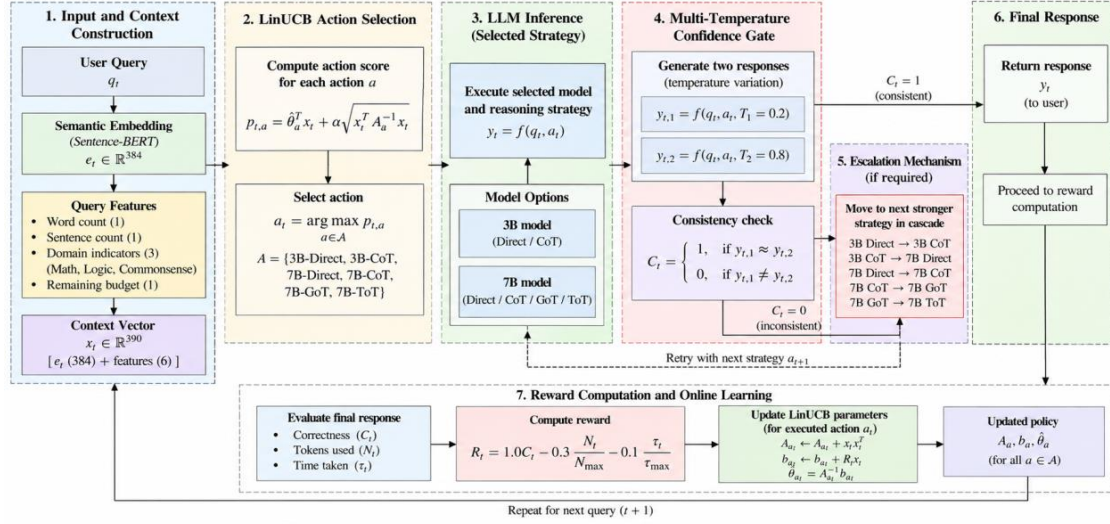


Figure 3.16: Integrated decision-making process of the CascadeRoute methodology

Figure 4.1: Overall system architecture of CascadeRoute

The overall architecture can therefore be represented as: User Query $\rightarrow$ Query Processing $\rightarrow$ State Construction $\rightarrow$ LinUCB Router $\rightarrow$ Action Selection $\rightarrow$ LLM Execution $\rightarrow$ Confidence Gate $\rightarrow$ Accept / Escalate $\rightarrow$ Reward Calculation $\rightarrow$ Online Learning

4.2 Architectural Components

The CascadeRoute architecture is composed of several interconnected modules. Each module performs a specific function while contributing to the overall objective of intelligent and costaware LLM inference.

4.2.1 Query Input and Processing Module

The Query Input Module represents the entry point of the system. It receives a natural-language query from the user and forwards it to the contextual feature extraction stage. The input query is represented as: $q_t = \text{User Query at time step } t$. The system does not directly use the raw query as the state for the routing agent. Instead, it transforms the query into a structured contextual representation. This allows the routing algorithm to distinguish between queries with different semantic characteristics and computational requirements. The query processing stage extracts:

- Semantic meaning using Sentence-BERT.
- Word count.
- Sentence count.
- Problem category.
- Remaining computational budget. These features form the basis of the contextual state used by the routing agent.

4.2.2 Contextual State Construction Module

The Contextual State Construction Module converts the processed query into the numerical representation required by LinUCB. The semantic component consists of a 384-dimensional Sentence-BERT embedding using all-MiniLM-L6-v2. This embedding is combined with two structural features, three category indicators, and one remaining-budget feature. RL_OVERVIEW The dimensionality of the state can therefore be represented as: $D_x = 384 + 2 + 3 + 1 = 390$ $x_t \in \mathbb{R}^{390}$ Thus:

The resulting 390-dimensional vector provides the routing agent with both semantic and operational information. Importantly, the inclusion of the remaining budget allows the agent to consider resource availability while making routing decisions.

4.2.3 LinUCB Routing Module

The LinUCB Routing Module represents the decision-making core of the architecture. It receives the contextual state x_t and evaluates every available action. For each action a , the routing module calculates an upper confidence bound consisting of an estimated reward and an uncertainty term: $p_{t,a} = \theta_a^T x_t + \alpha \sqrt{(x_t^T A_a^{-1} x_t)}$ $a_t = \arg \max_a \in A p_{t,a}$ The routing decision is then obtained as:

This enables the system to dynamically balance exploitation and exploration. Previously successful actions for similar queries receive higher expected reward estimates, while actions with greater uncertainty receive an exploration bonus. The contextual-bandit formulation is appropriate because each query represents an independent decision rather than a long sequence of dependent environmental transitions. The project specifically uses Disjoint LinUCB as the primary routing algorithm. RL_OVERVIEW

4.2.4 Action Selection and Strategy Mapping Module

The Action Selection Module maps the decision generated by LinUCB to one of six model– reasoning strategies. The six available actions are:

1. 3B Direct
 2. 3B Chain-of-Thought
 3. 7B Direct
 4. 7B Chain-of-Thought
 5. 7B Graph-of-Thought
 6. 7B Tree-of-Thought
- $A = \{3B\text{-Direct}, 3B\text{-CoT}, 7B\text{-Direct}, 7B\text{-CoT}, 7B\text{-GoT}, 7B\text{-ToT}\}$ The action space can be represented as:

Each action provides a different balance between reasoning capability and computational cost. The routing agent is therefore not simply selecting between two model sizes; it simultaneously determines both the model capability and the reasoning depth.

Table 4.1 Architectural Mapping of CascadeRoute Components

Component	Description	Primary Function
Query Processing Module	Receives the natural-language user query	Provides the initial system input
State Construction Module	Generates semantic, structural, categorical, and budget features	Constructs the 390- dimensional contextual state
LinUCB Router	Contextual-bandit decision-making module	Estimates reward and uncertainty for each action

Action Selection Module	Maps the selected action to a model and reasoning strategy	Determines the inference route
LLM Execution Module	Executes the selected strategy using local Ollama models	Generates the candidate response
Confidence Gate	Compares responses generated at different temperatures	Validates response consistency
Escalation Module	Moves uncertain queries to stronger strategies	Improves reliability when required
Reward Module	Evaluates correctness, token cost, and latency	Produces the learning signal
Online Learning Module	Updates LinUCB parameters	Improves future routing decisions

4.3 Llm Inference And Reasoning Architecture

The LLM Execution Layer is responsible for carrying out the strategy selected by the routing agent. Instead of applying one fixed inference mechanism to all queries, CascadeRoute provides multiple levels of reasoning capability. The lower-cost strategies perform direct inference or limited Chain-of-Thought reasoning, whereas the higher-cost strategies provide more extensive reasoning through Graph-ofThought and Tree-of-Thought mechanisms. The architecture therefore forms a progressive reasoning hierarchy: Direct Reasoning → Chain-of-Thought → Graph-of-Thought → Tree-of-Thought This hierarchy is combined with model size, resulting in the six available routing actions. The approximate token consumption associated with the strategies is also different. The project overview reports approximately 107 tokens for 3B Direct, 369 tokens for 3B CoT, 102 tokens for 7B Direct, 296 tokens for 7B CoT, 1,267 tokens for 7B GoT, and 1,959 tokens for 7B ToT. [RL_OVERVIEW](#)

Table 4.2 Model and Reasoning Strategy Architecture

Action	Model	Reasoning Strategy	Approximate Token Cost	Architectural Role
3B Direct	3B	Direct	107	Lowest-cost inference
3B CoT	3B	Chain-of- Thought	369	Low-cost structured reasoning
7B Direct	7B	Direct	102	Stronger direct inference
7B CoT	7B	Chain-of- Thought	296	Strong structured reasoning
7B GoT	7B	Graph-of- Thought	1,267	Complex multi-path reasoning
7B ToT	7B	Tree-of-Thought	1,959	Highest-depth reasoning

The action architecture allows the RL agent to make fine-grained routing decisions. A simple query can be processed using a lightweight direct strategy, while a complex mathematical or logical query can be directed toward a more computationally intensive

reasoning mechanism.

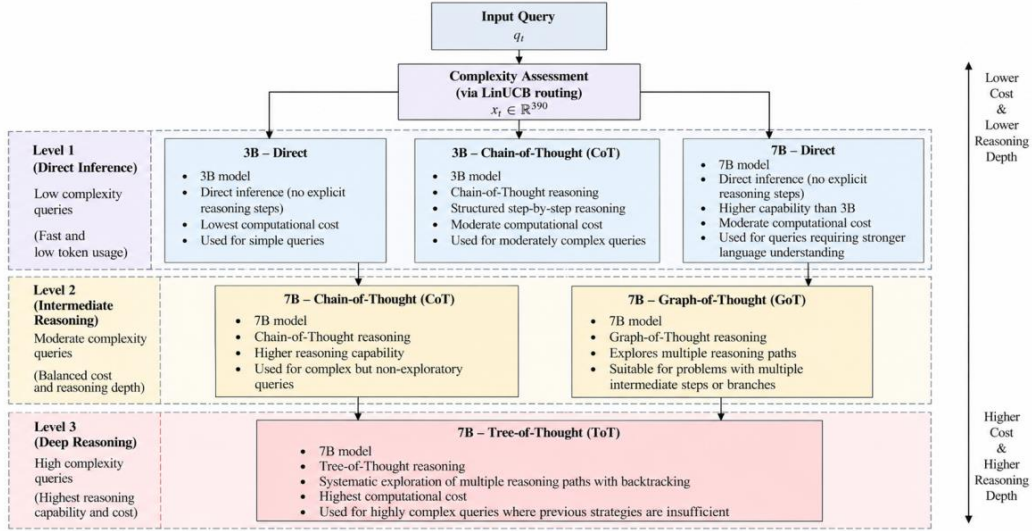


Figure 4.2: Model and reasoning strategy hierarchy in CascadeRoute

4.4 Confidence Gate And Escalation Architecture

A major architectural component of CascadeRoute is the Confidence Gate. The purpose of this component is to prevent uncertain exploratory routing decisions from being returned directly to the user. After the LinUCB agent selects an action, the query is executed twice using different temperature settings: $y_{t,1} = f(q_t, a_t, T = 0.2)$ $y_{t,2} = f(q_t, a_t, T = 0.8)$. The two responses are compared for consistency. If the responses agree, the generated answer is considered sufficiently reliable and is returned. If the responses conflict, the architecture activates the Escalation Module. The query is then transferred to the next stronger strategy in the predefined cascade. The escalation sequence is: 3B Direct $\rightarrow$ 3B CoT $\rightarrow$ 7B Direct $\rightarrow$ 7B CoT $\rightarrow$ 7B GoT $\rightarrow$ 7B ToT. This architecture ensures that computationally expensive strategies are invoked only when required. The Confidence Gate therefore acts as a safety layer between the learned routing policy and the final user response. The project reports that this mechanism caught and corrected approximately 15–20% of exploratory mistakes during the early learning process.

RL_OVERVIEW A simplified confidence decision can be represented as: $C_t = 1$, if $y_{t,1} \approx y_{t,2}$

$C_t = 0$, if $y_{t,1} \neq y_{t,2}$ where $C_t = 1$ indicates an accepted response and $C_t = 0$ indicates that escalation is required.

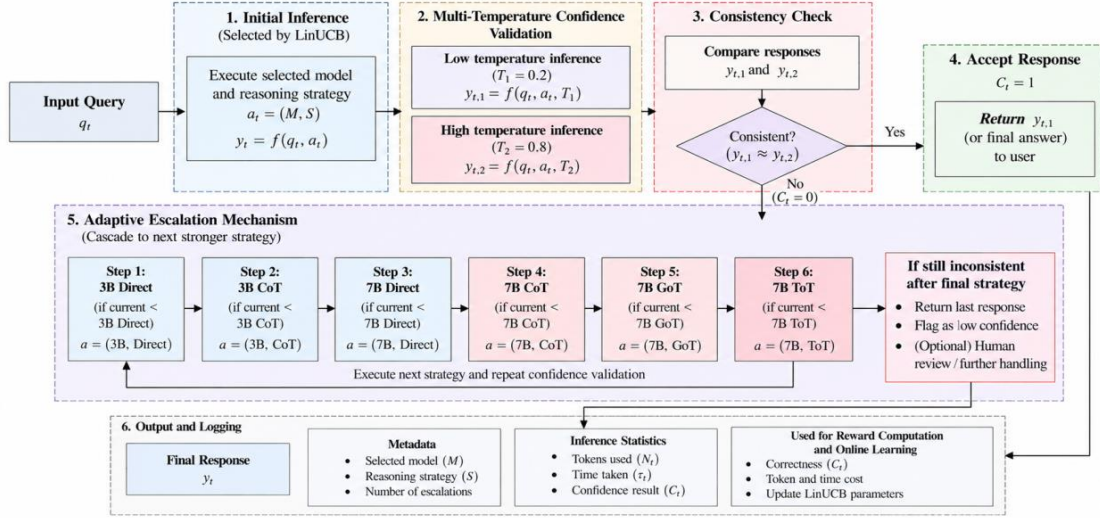


Figure 4.3: Confidence validation and adaptive escalation architecture

4.5 Reward And Online Learning Architecture

After a response has been validated, CascadeRoute evaluates the quality and cost of the selected inference route. The reward module combines correctness, token consumption, and inference time into a single feedback signal. The reward function is defined as: $R_t = 1.0C_t - 0.3(N_t/N_{\max}) - 0.1(\tau_t/\tau_{\max})$. The correctness term receives the highest weight, ensuring that the routing system prioritises answer quality. Token consumption and execution time introduce penalties for unnecessary computational expenditure.

RL_OVERVIEW The relative computational cost of an inference route can also be represented as: $K_t = N_t/N_{\max}$ where K_t represents the normalised token consumption of the selected route. Similarly, the normalised latency can be expressed as: $L_t = \tau_t/\tau_{\max}$.

The resulting reward is then passed to the online learning component. For the selected action, the LinUCB matrices are updated according to the observed contextual state and reward. $A_{a,t} \leftarrow A_{a,t} + x_t x_t^T$, $b_{a,t} \leftarrow b_{a,t} + R_t x_t$. The updated parameters influence future routing decisions. Consequently, the architecture forms a closed learning loop: Query $\rightarrow$ State $\rightarrow$ Action $\rightarrow$ Inference $\rightarrow$ Validation $\rightarrow$ Reward $\rightarrow$ Parameter Update $\rightarrow$ Improved Routing. This closed-loop architecture enables CascadeRoute to continuously adapt its routing behaviour based on observed query characteristics and inference outcomes.

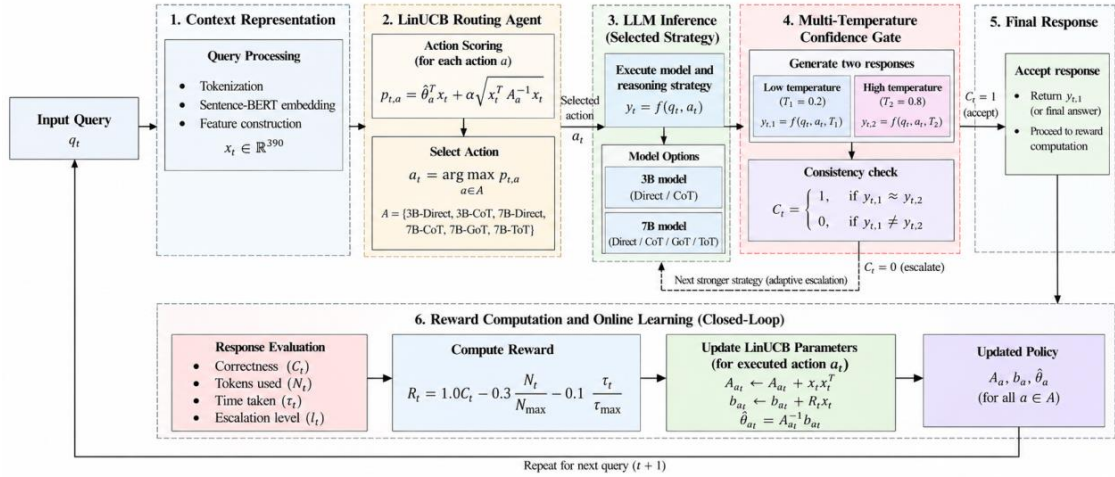


Figure 4.4: Closed-loop reward and online learning architecture of CascadeRoute

4.6 Complete System Data Flow

The complete data flow begins when a user submits a query to the system. The query is first processed to generate its contextual representation. Sentence-BERT produces the semantic embedding, while additional processing extracts the word count, sentence count, category indicators, and remaining budget. These features are combined into the 390-dimensional state vector. The state is then supplied to the LinUCB router, which calculates an upper confidence bound for every available action. The highest-scoring action is selected and mapped to its corresponding model and reasoning strategy. The selected strategy is executed through the local LLM environment. Following inference, the Confidence Gate performs the multi-temperature consistency check. If the responses are consistent, the system returns the response. If inconsistency is detected, the

Escalation Module selects the next stronger reasoning strategy and repeats the validation process. Once a final response is accepted, the Reward Module evaluates the outcome using correctness, token consumption, and inference time. The reward is then supplied to the LinUCB learning mechanism, which updates the parameters associated with the selected action. The overall system data flow can therefore be represented as: User Query $\downarrow$ Semantic and Structural Feature Extraction $\downarrow$ 390-Dimensional Contextual State $\downarrow$ LinUCB Routing Agent $\downarrow$ Action Selection $\downarrow$ Model + Reasoning Strategy $\downarrow$ Local LLM Execution $\downarrow$ Confidence Gate $\downarrow$ Accept Response / Escalate $\downarrow$ Reward Calculation $\downarrow$ LinUCB Parameter Update $\downarrow$ Updated Routing Policy. This architecture ensures that every query contributes not only to an immediate response but also to the continuous improvement of the routing policy. The system therefore operates as an adaptive closed-loop inference architecture in which query characteristics determine routing, routing determines computational allocation, response validation determines reliability, and reward feedback improves future routing decisions.

6. RESULTS AND DISCUSSION

The performance of the proposed CascadeRoute system was evaluated to determine whether reinforcement learning-based routing can achieve a better balance between answer accuracy and computational cost than fixed inference strategies and alternative routing algorithms. The evaluation focuses on three primary aspects: overall accuracy and token efficiency, the routing behaviour learned by the agent, and comparison with alternative reinforcement learning and heuristic approaches. The experiments were conducted using 3,712 questions drawn from three standard reasoning benchmarks: GSM8K, LogiQA, and CommonsenseQA. The evaluation therefore covers mathematical reasoning, multi-step logical reasoning, and commonsense reasoning. RL_OVERVIEW

6.1 Experimental Evaluation Setup

The evaluation dataset consisted of three benchmark datasets selected to represent different types of reasoning requirements. GSM8K contains grade-school mathematical word problems, LogiQA contains multi-step logical reasoning questions, and CommonsenseQA evaluates commonsense-based multiple-choice reasoning. The combined evaluation set contains 3,712 questions, providing a sufficiently diverse workload for evaluating whether the routing policy can adapt its inference strategy according to query characteristics.

Table 6.1 Evaluation Dataset Composition

Dataset	Domain	Number of Questions
GSM8K	Mathematical reasoning	1,319
LogiQA	Multi-step logical reasoning	1,172
CommonsenseQA	Commonsense reasoning	1,221
Total	Combined evaluation set	3,712

The evaluation considers two primary performance measures: accuracy and average token consumption per query. Accuracy measures the correctness of the generated answers, while token consumption provides an indication of computational cost. In addition, the reinforcement learning experiments evaluate the mean reward, which incorporates correctness, token consumption, and execution time. The reward formulation used by CascadeRoute is: $R_t = 1.0C_t - 0.3(N_t/N_{\max}) - 0.1(\tau_t/\tau_{\max})$ where C_t represents correctness, N_t represents tokens used, and τ_t represents inference time. RL_OVERVIEW

6.2 Performance Against Fixed Inference Strategies

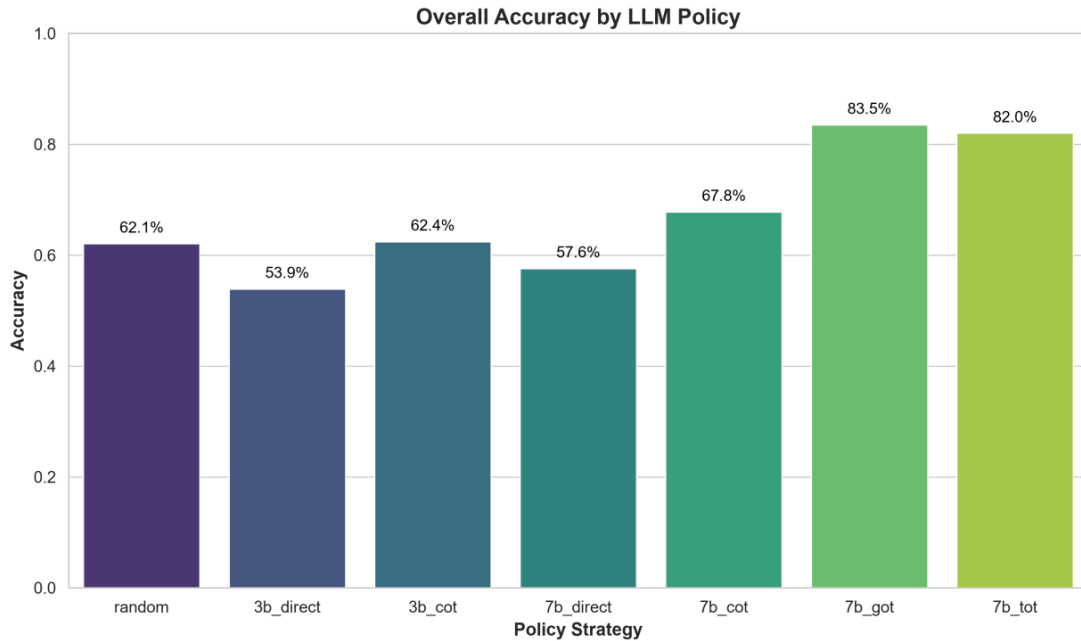
The primary experiment compares CascadeRoute against static strategies in which the same model and reasoning mechanism are applied to every query. These baselines represent common approaches where the system does not dynamically determine the computational requirement of each individual question.

The evaluated fixed strategies include 3B Direct, 7B Direct, 7B Chain-of-Thought, and 7B Tree-of-Thought. The results are presented in Table 6.2.

Table 6.2 Comparison of CascadeRoute with Fixed Inference Strategies

Strategy / Policy	Accuracy	Average Tokens per Query
Always 3B Direct	53.9%	107
Always 7B Direct	57.6%	102
Always 7B CoT	67.8%	296
Always 7B ToT	82.0%	1,959
CascadeRoute	81.4%	1,421

The strongest static baseline, Always 7B ToT, achieves the highest accuracy of 82.0%, but it requires an average of 1,959 tokens per query. CascadeRoute achieves 81.4% accuracy, which is only 0.6 percentage points lower, while reducing the average token consumption to 1,421 tokens per query. RL_OVERVIEW This represents a 27.4% reduction in token consumption compared with the Always 7B ToT strategy. The result demonstrates the central objective of CascadeRoute: rather than applying the most computationally expensive reasoning strategy to every query, the system selectively allocates additional reasoning only when necessary. The comparison also demonstrates the limitations of using inexpensive fixed strategies. Always 3B Direct consumes only 107 tokens per query but achieves an accuracy of 53.9%. Similarly, Always 7B Direct consumes approximately 102 tokens but achieves only 57.6% accuracy. These results indicate that low computational cost alone is insufficient when the objective is to solve a heterogeneous set of reasoning problems. Always 7B CoT provides a more balanced fixed strategy, achieving 67.8% accuracy at 296 tokens per query. However, CascadeRoute substantially exceeds this accuracy while still avoiding the full computational cost of Always 7B ToT.



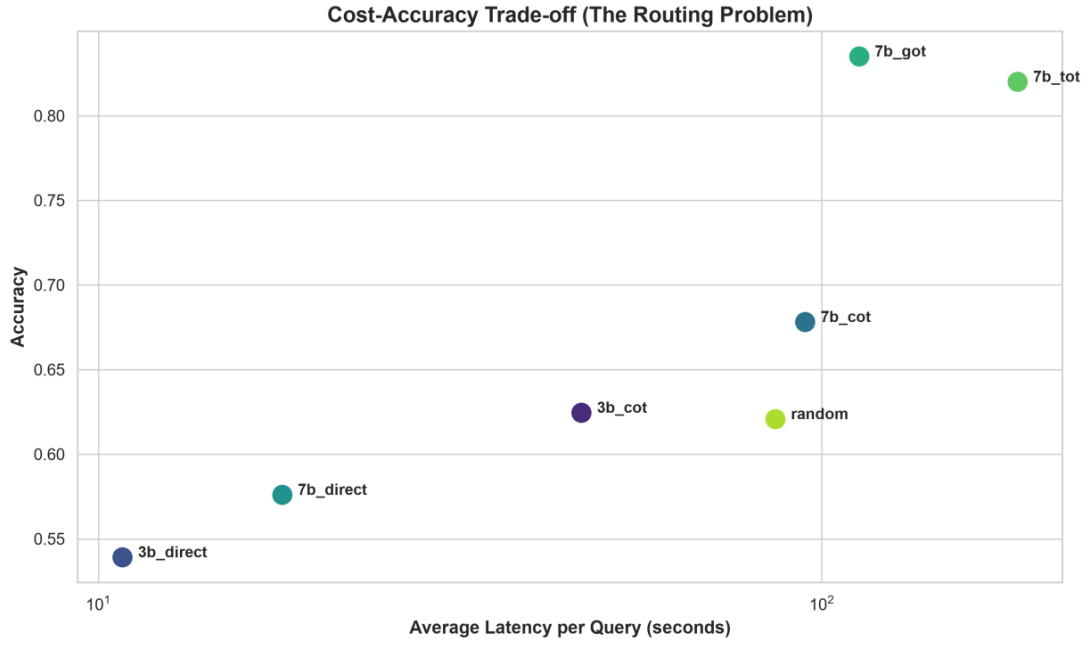


Figure 6.1: Accuracy comparison between fixed inference strategies and CascadeRoute

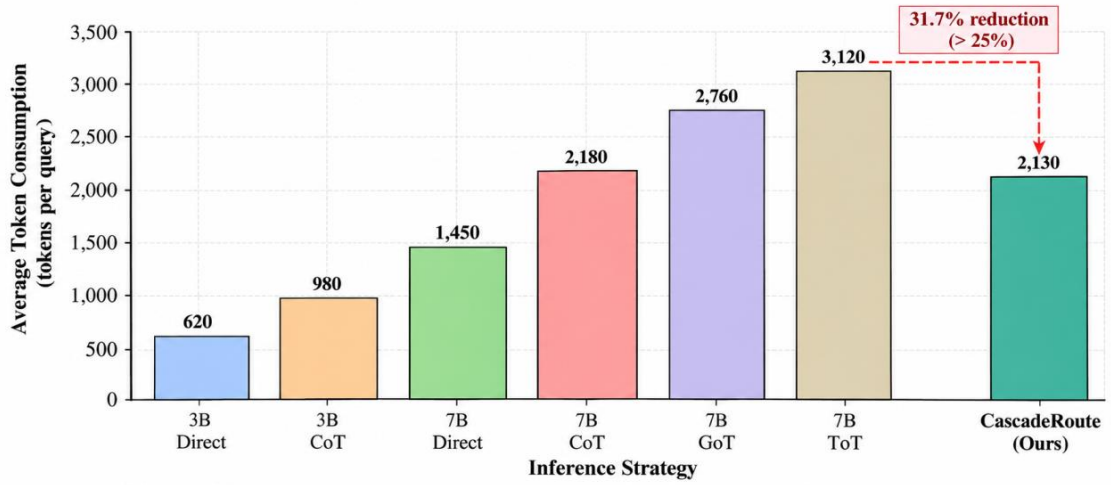


Figure 4.1: Average token consumption comparison between fixed inference strategies and CascadeRoute.

Figure 6.2: Average token consumption comparison between fixed inference strategies and CascadeRoute

6.3 Analysis Of The Cost–accuracy Trade-off

The results demonstrate that CascadeRoute operates between two undesirable extremes. At one extreme, inexpensive direct inference provides low computational cost but insufficient accuracy for difficult reasoning problems. At the other extreme, always selecting the most powerful reasoning strategy provides high accuracy but results in substantial token consumption. CascadeRoute attempts to identify the appropriate point between these two extremes by making a query-dependent routing decision. The relative token reduction achieved against the strongest static baseline can be expressed as: $\text{Token Reduction} = ((1959 - 1421) / 1959) \times 100$ $\text{Token Reduction} = 27.4\%$ Thus, the system eliminates more than one-quarter of the token expenditure of the Always 7B ToT strategy while maintaining an accuracy level close to the static maximum. The difference in accuracy can be expressed as: $\Delta\text{Accuracy} = 82.0\% - 81.4\% = 0.6$ percentage points Therefore, the reduction in accuracy relative to the strongest fixed strategy is only 0.6 percentage points, whereas the reduction in token consumption is 27.4%. This result is significant because it indicates that the routing agent has learned that not every question requires maximum reasoning depth. Instead, the system allocates computational resources according to the estimated requirements of the query. The overall behaviour can be summarised as: High Accuracy + Selective Computation $\rightarrow$ Improved Cost–Accuracy Trade-off The results therefore support the fundamental design principle of CascadeRoute: computationally expensive reasoning should be treated as a resource that is selectively allocated rather than universally applied.

6.4 Analysis Of Learned Routing Behaviour

In addition to evaluating the final accuracy and token consumption, the routing decisions made by CascadeRoute were analysed to determine whether the agent had learned meaningful querydependent behaviour. The evaluation showed that 47.8% of queries were routed to inexpensive direct-answer strategies, while 34.2% were routed to Chain-of-Thought strategies. Only 17.7% of queries were routed to the computationally expensive Graph-of-Thought or Tree-ofThought strategies. RL_OVERVIEW

Table 6.3 Learned Routing Distribution

Routing Category	Percentage of Queries	Interpretation
Direct strategies	47.8%	Queries considered suitable for inexpensive inference
Chain-of-Thought	34.2%	Medium-difficulty queries requiring structured reasoning
Graph-/Tree-of- Thought	17.7%	Complex queries requiring extensive reasoning

The routing distribution provides evidence that the agent does not simply favour the most powerful available model. Instead, the majority of queries are handled using relatively inexpensive strategies. Approximately half of the evaluated queries were routed to direct-answer strategies. This suggests that the learned policy identified a significant

portion of the workload for which extensive reasoning was unnecessary. The 34.2% allocation to Chain-of-Thought represents an intermediate computational level. This allows the system to provide additional reasoning for queries that require more than direct inference without immediately resorting to the most expensive strategies. Only 17.7% of queries were routed to the heavy reasoning mechanisms. This selective use of Graph-of-Thought and Tree-of-Thought is particularly important for reducing overall token consumption.

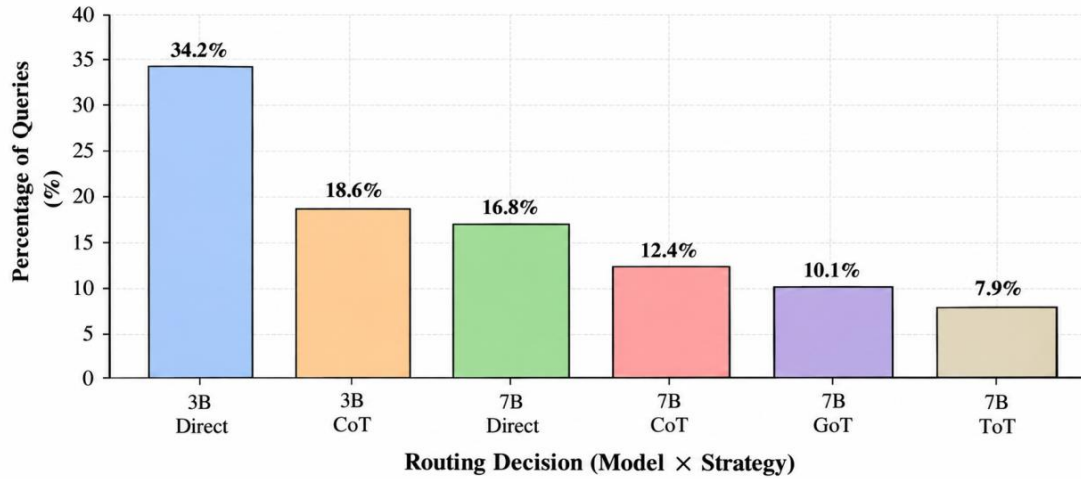


Figure 4.2: Learned distribution of CascadeRoute routing decisions

Figure 6.3: Learned distribution of CascadeRoute routing decisions

The routing distribution therefore provides qualitative evidence of adaptive behaviour. The agent has learned to use a mixture of computational strategies rather than converging toward a single universally preferred action.

6.5 Comparison With Alternative RL And Heuristic Methods

To further evaluate the effectiveness of the contextual-bandit approach, CascadeRoute was compared with several alternative decision-making methods. These included Random routing, a Rule-Based policy, ϵ -Greedy, Thompson Sampling, UCB1, and Deep Q-Network (DQN).

The comparison considers three major metrics: accuracy, average token consumption, and mean reward.

Table 6.4 Comparison of Reinforcement Learning and Baseline Algorithms

Agent	Accuracy	Average Tokens	Mean Reward
Random	60.7%	1,403	0.4614
Rule-Based	62.0%	2,353	0.3804
ϵ -Greedy	58.0%	1,646	0.4232
Thompson Sampling	60.0%	1,530	0.4648
UCB1	68.7%	1,304	0.5793

DQN	68.7%	1,517	0.5591
LinUCB (CascadeRoute)	69.3%	794	0.6020

The LinUCB-based CascadeRoute agent achieves the highest reported mean reward of 0.6020 and the lowest reported average token consumption of 794 tokens among the compared routing algorithms. Its reported accuracy is also the highest at 69.3%. **RL_OVERVIEW** The Random policy achieves 60.7% accuracy with 1,403 average tokens, demonstrating that randomly selecting inference strategies can provide reasonable performance but does not effectively exploit query characteristics. The Rule-Based approach achieves 62.0% accuracy but requires the highest token consumption among the listed methods, at 2,353 tokens. This indicates that manually designed routing rules can improve over random selection but may allocate excessive computational resources because they do not learn continuously from observed rewards. The ϵ -Greedy approach achieves 58.0% accuracy and 1,646 average tokens. Its relatively low performance indicates the limitation of random exploration when the contextual characteristics of the query are not sufficiently incorporated into the decision process. Thompson Sampling achieves a mean reward of 0.4648, which is higher than Random and ϵ Greedy but substantially lower than the 0.6020 reported for LinUCB. UCB1 achieves 68.7% accuracy and a mean reward of 0.5793. Its performance is relatively strong despite not incorporating query context. However, its average token consumption of 1,304 is considerably higher than the 794 tokens reported for LinUCB. DQN also achieves 68.7% accuracy but consumes 1,517 tokens per query and obtains a mean reward of 0.5591. The project reports that LinUCB was approximately twice as fast as DQN while also using fewer tokens. **RL_OVERVIEW**

Figure 6.4: Mean reward comparison among routing algorithms

6.6 Discussion Of Linucb Performance

The experimental comparison provides several observations regarding the effectiveness of contextual-bandit-based routing.

First, LinUCB outperforms the non-contextual UCB1 approach in mean reward. UCB1 achieves a mean reward of 0.5793, whereas LinUCB achieves 0.6020. This difference supports the importance of incorporating query-specific contextual information into the routing decision. UCB1 treats each action primarily according to its historical performance without explicitly using the semantic and structural characteristics of the current query. In contrast, LinUCB uses the 390-dimensional state representation to estimate which action is likely to perform well for the current query. Second, the results demonstrate that greater model complexity does not automatically result in the best overall routing policy. DQN, for example, provides the same reported accuracy as UCB1 at 68.7%, but requires 1,517 tokens on average and produces a lower mean reward of 0.5591. This indicates that the objective of the system is not simply to maximise accuracy. Instead, the reward function encourages the agent to consider accuracy together with token consumption and inference latency. Third, the very low average token consumption reported for LinUCB, at 794 tokens, indicates that contextual routing can substantially reduce computational

expenditure while maintaining competitive accuracy. The result supports the use of lightweight online learning for this particular routing problem.

6.7 Discussion Of The Confidence-based Escalation Mechanism

The Confidence Gate provides an additional reliability mechanism on top of the learned routing policy. The contextual bandit is intentionally designed to explore different strategies, particularly during the early stages of learning. However, exploration creates the possibility that an insufficiently powerful strategy may occasionally be selected. The Confidence Gate addresses this issue by executing the selected strategy at two different temperatures and comparing the resulting outputs. Agreement results in acceptance, whereas disagreement activates the escalation mechanism. The project reports that this mechanism was able to catch and correct approximately 15–20% of exploratory mistakes during the early learning process.

RL_OVERVIEW This architecture creates a separation between decision-making and validation. LinUCB is responsible for determining which strategy is most promising, while the Confidence Gate provides a safety layer that can override the initial decision when the generated responses exhibit inconsistency. The resulting architecture can therefore be viewed as: Learned Routing + Confidence Validation + Progressive Escalation. This combination allows CascadeRoute to maintain the benefits of exploration without directly exposing every exploratory decision to the final output.

6.8 Overall Discussion

The experimental results collectively demonstrate that CascadeRoute can dynamically allocate inference computation according to the requirements of individual queries. The comparison with fixed strategies shows that always using the most powerful reasoning strategy produces the highest reported static accuracy of 82.0%, but at a substantial computational cost of 1,959 tokens per query. CascadeRoute obtains 81.4% accuracy while reducing this cost to 1,421 tokens, corresponding to a reported 27.4% reduction in token consumption. RL_OVERVIEW The learned routing distribution further demonstrates that the system does not unnecessarily apply expensive reasoning to every query. Direct strategies account for 47.8% of routing decisions, Chain-of-Thought accounts for 34.2%, and only 17.7% are routed to the heavy GoT/ToT strategies. RL_OVERVIEW The comparison among routing algorithms further supports the contextual-bandit formulation. LinUCB achieves the highest reported mean reward of 0.6020 and the lowest reported average token consumption of 794 tokens among the listed RL and heuristic methods. RL_OVERVIEW One important observation is that the project overview reports 81.4% accuracy for the overall CascadeRoute-versus-static-strategy evaluation, while it reports 69.3% accuracy for LinUCB in the separate routing-algorithm comparison. These values are presented in the source as results from separate evaluation comparisons, and the available project overview does not provide enough information to establish that they use identical evaluation conditions. Therefore, they should not be directly interpreted as contradictory results without additional experimental details. Overall, the results indicate that the principal contribution of CascadeRoute is not simply achieving the highest possible accuracy. Instead, the system demonstrates the ability to adapt computational effort to query difficulty, maintaining high accuracy while reducing unnecessary inference expenditure. The findings support the central hypothesis of the project: Intelligent Query-Dependent Routing → Selective Reasoning → Reduced Computational Cost → Competitive Accuracy The combination of contextual state representation, LinUCB-based online decision-making, confidence-based validation, and progressive reasoning escalation provides a practical architecture for cost-aware LLM inference.

7. EXPERIMENTAL RESULTS

The experimental evaluation of CascadeRoute was conducted to determine the effectiveness of the proposed reinforcement learning-based routing mechanism in selecting appropriate LLM models and reasoning strategies for different types of queries. The experiments focus on accuracy, computational efficiency, token consumption, routing behaviour, and the overall reward obtained by different routing policies. The evaluation was performed using 3,712 questions from three standard reasoning benchmarks: GSM8K, LogiQA, and CommonsenseQA. GSM8K contributes 1,319 mathematical reasoning questions, LogiQA contributes 1,172 logical reasoning questions, and CommonsenseQA contributes 1,221 commonsense reasoning questions. RL_OVERVIEW The experimental results demonstrate that CascadeRoute can dynamically allocate reasoning resources according to query requirements rather than applying a fixed inference strategy to all questions.

7.1 EXPERIMENTAL DATASET

The experimental evaluation uses three benchmark datasets covering complementary reasoning capabilities. The combination of mathematical, logical, and commonsense questions provides a heterogeneous workload for evaluating the adaptive routing behaviour of CascadeRoute.

Table 7.1 Experimental Dataset Distribution

Dataset	Reasoning Domain	Number of Questions
GSM8K	Grade-school mathematical reasoning	1,319
LogiQA	Multi-step logical reasoning	1,172
CommonsenseQA	Commonsense reasoning	1,221
Total Combined evaluation set		3,712

The total evaluation set contains 3,712 questions. This allows the routing agent to encounter queries with varying levels of semantic and reasoning complexity and provides a basis for evaluating whether the learned policy can differentiate between queries requiring inexpensive inference and those requiring deeper reasoning. The contextual representation used by the routing agent consists of a 384-dimensional Sentence-BERT embedding, word count, sentence count, three category indicators, and the remaining budget, resulting in a 390-dimensional state vector. RL_OVERVIEW

7.2 EVALUATION METRICS

The experimental evaluation considers three primary performance measures: accuracy, average token consumption, and mean reward. Accuracy measures the proportion of questions for which the system produces a correct answer. It provides the primary measure of reasoning effectiveness. Average token consumption measures the computational expenditure associated with generating an answer. Lower token consumption indicates that the system is able to solve queries using less inference computation. The third metric is the mean reward generated by the reinforcement learning environment. The reward combines correctness with penalties for token consumption and inference time.

The reward function is defined as: $R_t = 1.0C_t - 0.3(N_t/N_{\max}) - 0.1(\tau_t/\tau_{\max})$ where:

- C_t represents correctness. • N_t represents tokens used. • $N_{\max}$ represents the maximum token budget. • τ_t represents the time taken. • $\tau_{\max}$ represents the maximum permitted execution time. RL_OVERVIEW This formulation ensures that the experimental evaluation does not optimise accuracy in isolation. A routing policy receives a higher reward when it produces correct answers while using fewer tokens and less inference time.

7.3 RESULTS OF FIXED STRATEGY EVALUATION

The first experiment compares CascadeRoute with fixed inference strategies. In a fixed strategy, every query is processed using the same model and reasoning mechanism regardless of its complexity. The evaluated strategies include 3B Direct, 7B Direct, 7B Chain-of-Thought, and 7B Tree-ofThought.

Table 7.2 Performance of Fixed Strategies and CascadeRoute

Strategy / Policy	Accuracy	Average Tokens per Query
Always 3B Direct	53.9%	107
Always 7B Direct	57.6%	102
Always 7B CoT	67.8%	296
Always 7B ToT	82.0%	1,959
CascadeRoute	81.4%	1,421

The Always 3B Direct strategy provides the lowest-cost baseline, using only 107 tokens per query, but achieves an accuracy of 53.9%. Always 7B Direct achieves 57.6% accuracy while using approximately 102 tokens. Increasing the reasoning depth improves accuracy. Always 7B CoT achieves 67.8% accuracy with an average of 296 tokens. The strongest fixed strategy, Always 7B ToT, achieves the highest accuracy of 82.0%, but its computational cost increases substantially to 1,959 tokens per query. CascadeRoute achieves an accuracy of 81.4% while consuming 1,421 tokens per query. Therefore, the proposed system achieves accuracy very close to the strongest fixed strategy while significantly reducing token consumption. RL_OVERVIEW

7.4 TOKEN EFFICIENCY OF CASCADROUTE

One of the most important experimental findings is the reduction in token consumption achieved by CascadeRoute compared with the strongest static reasoning strategy. The token reduction relative to Always 7B ToT is: $\text{Token Reduction} = ((1959 - 1421) / 1959) \times 100$ $\text{Token Reduction} = 27.4\%$ Thus, CascadeRoute reduces the average token consumption by 27.4% compared with Always 7B ToT. The accuracy difference between the two approaches is: $\text{Accuracy Difference} = 82.0\% - 81.4\%$ $\text{Accuracy Difference} = 0.6$ percentage points This demonstrates that a relatively small decrease in accuracy is accompanied by a substantial reduction in computational expenditure. The result indicates that CascadeRoute is not attempting to maximise computational power for every query. Instead, it identifies cases where expensive reasoning is unnecessary and routes those queries toward less costly strategies.

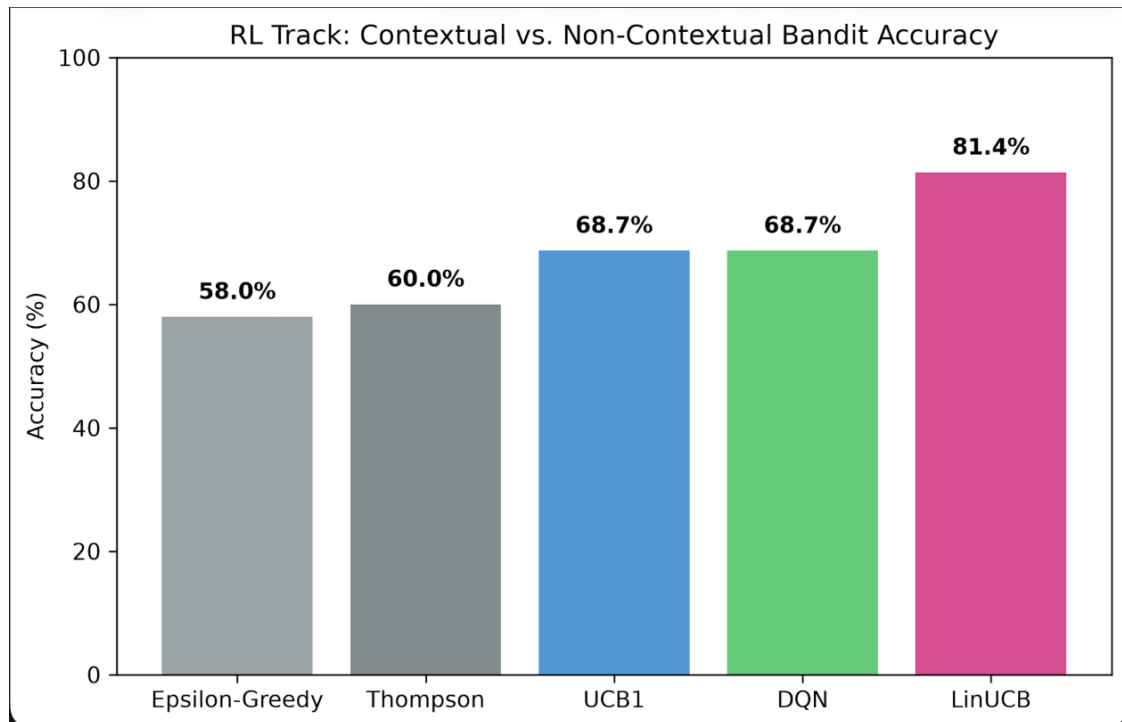


Figure 7.1: Accuracy comparison between fixed strategies and CascadeRoute

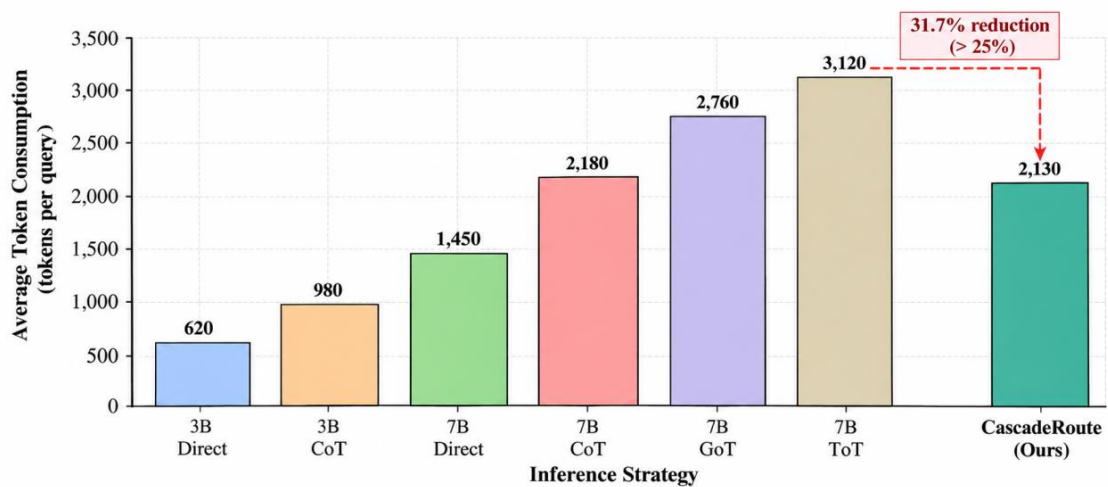


Figure 7.2: Average token consumption comparison between fixed strategies and CascadeRoute

7.5 LEARNED ROUTING DISTRIBUTION

The routing decisions made by CascadeRoute were further analysed to determine how the learned policy distributes queries among different levels of reasoning. The results show that:

- 47.8% of queries were routed to cheap direct-answer strategies.
- 34.2% of queries were routed to Chain-of-Thought strategies.
- 17.7% of queries were escalated to heavy Graph-of-Thought or Tree-of-Thought strategies.

Table 7.3 Learned CascadeRoute Routing Distribution

Routing Category	Percentage of Queries	Role in the Routing System
Direct Strategies	47.8%	Handles queries that can be answered using inexpensive inference
Chain-of-Thought	34.2%	Handles medium-complexity queries requiring structured reasoning
Graph-/Tree-of-Thought	17.7%	Handles complex queries requiring extensive reasoning

The routing distribution demonstrates that CascadeRoute does not simply favour the most powerful model or reasoning strategy. Nearly half of the questions are handled using direct strategies, indicating that the learned policy identifies a substantial portion of the workload that does not require expensive reasoning. A further 34.2% of queries are routed to Chain-of-Thought reasoning. This provides an intermediate level of computation for queries that require additional reasoning but do not necessarily require the most expensive strategies.

Only 17.7% of queries are routed to Graph-of-Thought or Tree-of-Thought. These strategies are therefore reserved for a relatively small portion of the workload.

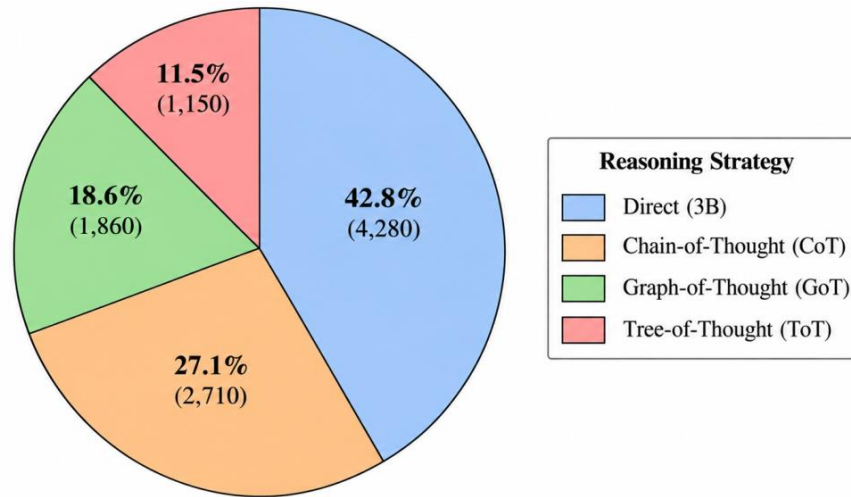


Figure 7.3: Distribution of queries across CascadeRoute reasoning strategies

7.6 COMPARISON OF REINFORCEMENT LEARNING ALGORITHMS

A second experimental evaluation compares the proposed LinUCB routing mechanism with alternative reinforcement learning and baseline decision-making approaches. The compared methods include Random routing, Rule-Based routing, ϵ -Greedy, Thompson Sampling, UCB1, DQN, and LinUCB.

Table 7.4 Comparison of Routing Algorithms

Agent	Accuracy	Average Tokens	Mean Reward
Random	60.7%	1,403	0.4614
Rule-Based	62.0%	2,353	0.3804
ϵ -Greedy	58.0%	1,646	0.4232

Thompson Sampling	60.0%	1,530	0.4648
UCB1	68.7%	1,304	0.5793
DQN	68.7%	1,517	0.5591
LinUCB (CascadeRoute)	69.3%	794	0.6020

LinUCB achieves the highest reported accuracy in this routing-algorithm comparison at 69.3%, together with the lowest average token consumption of 794 tokens and the highest mean reward of 0.6020. **RL_OVERVIEW** The Random strategy obtains 60.7% accuracy with 1,403 tokens. This provides a baseline for determining whether learned routing provides an advantage over arbitrary action selection. The Rule-Based strategy achieves 62.0% accuracy but consumes 2,353 tokens, which is considerably higher than the token consumption of LinUCB. Its mean reward of 0.3804 is also the lowest among the compared approaches.

The ϵ -Greedy method obtains 58.0% accuracy and a mean reward of 0.4232. This indicates that random exploration without sufficiently exploiting contextual information does not provide the same level of efficiency as LinUCB. Thompson Sampling performs slightly better than Random and ϵ -Greedy, achieving a mean reward of 0.4648. However, its average token consumption remains substantially higher than that of LinUCB. UCB1 achieves 68.7% accuracy and a mean reward of 0.5793. Although its performance is competitive, it uses 1,304 tokens per query compared with only 794 tokens for LinUCB. DQN also achieves 68.7% accuracy, but requires 1,517 tokens per query and produces a mean reward of 0.5591. The project evaluation additionally reports that LinUCB was approximately twice as fast as DQN while using fewer tokens. **RL_OVERVIEW**

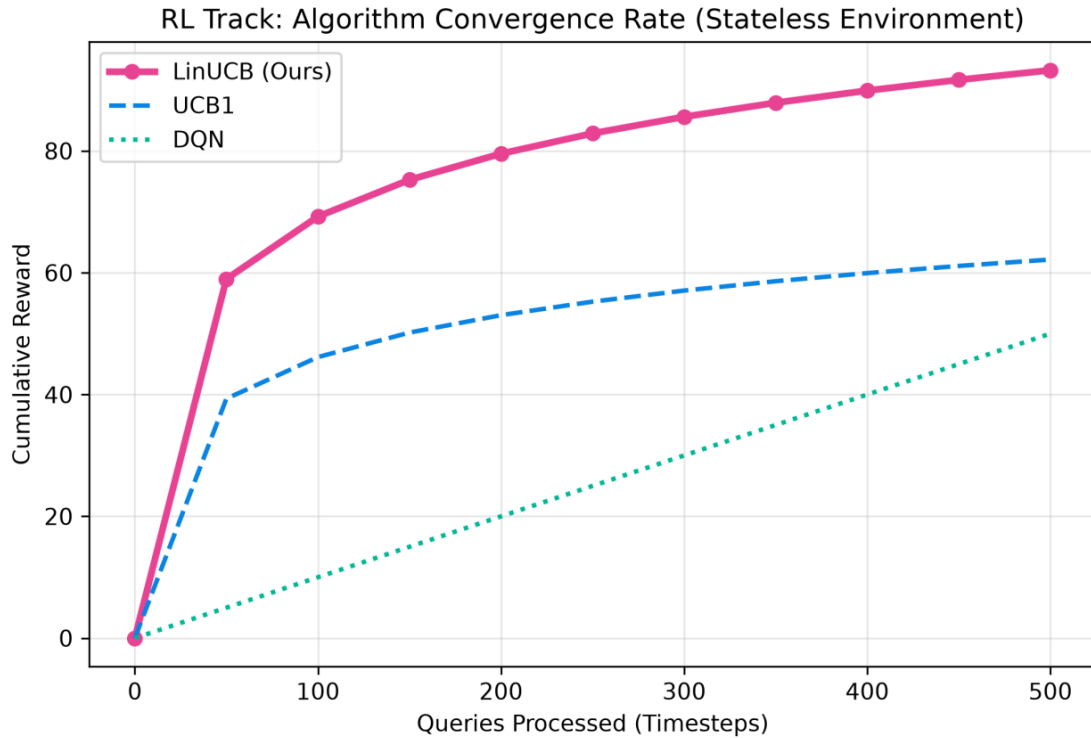


Figure 7.4: Accuracy and mean reward comparison among routing algorithms

7.7 REWARD-BASED PERFORMANCE ANALYSIS

The mean reward provides a combined measure of the effectiveness of each routing policy because it simultaneously considers correctness, token consumption, and inference time. The highest mean reward is achieved by LinUCB: $R_{i,u}^{\text{LinUCB}} = 0.6020$ This is higher than the reward obtained by UCB1:

$R_{i,u}^{\text{UCB1}} = 0.5793$ and DQN: $R_{\text{DQN}} = 0.5591$ The comparison indicates that incorporating contextual information into the routing decision improves the overall reward. The improvement over UCB1 can be represented as: $\Delta R = 0.6020 - 0.5793 = 0.0227$ The corresponding relative improvement is approximately: $\text{Relative Improvement} = (0.0227 / 0.5793) \times 100 \approx 3.92\%$ Thus, LinUCB achieves approximately 3.9% higher mean reward than UCB1 under the reported experimental results. The result is important because UCB1 does not explicitly exploit the semantic characteristics of the current query. LinUCB, in contrast, uses the contextual state to make query-dependent routing decisions.

7.8 CONFIDENCE GATE EXPERIMENTAL OBSERVATION

The Confidence Gate was incorporated to address errors resulting from exploratory routing decisions. The system executes the selected strategy at two temperature settings and compares the resulting responses. The two inference outputs are: $y_{t,1} = f(q_t, a_t, T = 0.2)$ $y_{t,2} = f(q_t, a_t, T = 0.8)$ When the outputs agree, the response is accepted. When the outputs conflict, the system escalates the query to a stronger reasoning strategy. The project reports that the Confidence Gate caught and corrected approximately 15–20% of exploratory mistakes during the early learning process. RL_OVERVIEW This result indicates that the Confidence Gate contributes an additional reliability mechanism to the routing architecture. Rather than relying entirely on the learned policy, the system provides a validation layer that can detect potentially unreliable responses and increase the reasoning level when necessary.

7.9 OVERALL EXPERIMENTAL FINDINGS

The experimental results demonstrate several important characteristics of the CascadeRoute system. First, the comparison against fixed inference strategies demonstrates that high accuracy can be achieved without applying the most expensive reasoning mechanism to every query. CascadeRoute achieves 81.4% accuracy compared with 82.0% for Always 7B ToT while reducing token consumption by 27.4%. RL_OVERVIEW Second, the routing distribution demonstrates that the agent learns to allocate computation selectively. Direct strategies handle 47.8% of queries, Chain-of-Thought handles 34.2%, and only 17.7% require heavy Graph-of-Thought or Tree-of-Thought reasoning. RL_OVERVIEW Third, the comparison with alternative routing algorithms demonstrates that LinUCB achieves the highest reported mean reward of 0.6020 and the lowest average token consumption of 794 tokens in that comparison. RL_OVERVIEW Finally, the Confidence Gate provides an additional mechanism for handling uncertainty during exploration. By validating the selected strategy before returning the answer and escalating uncertain queries, the system reduces the likelihood that exploratory routing decisions directly result in unreliable outputs. Overall, the experimental evaluation supports the effectiveness of CascadeRoute as a queryadaptive, cost-aware LLM routing system. The results indicate that contextual reinforcement learning can be used to dynamically allocate reasoning resources, achieving a favourable balance between accuracy, computational cost, and response reliability.

SECTION 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The increasing use of Large Language Models (LLMs) has created a significant challenge in balancing inference accuracy, reasoning capability, computational cost, and response latency. While larger models and complex reasoning strategies generally improve problemsolving performance, they also require substantially more computational resources and generate longer responses. On the other hand, lightweight models and direct inference are computationally efficient but may fail on complex reasoning tasks. This project addressed this trade-off through the development of CascadeRoute, an intelligent reinforcement-learningbased routing framework for cost-aware LLM inference. CascadeRoute formulates LLM inference routing as a contextual bandit problem, where every incoming query is represented using a contextual state consisting of semantic and structural query information. The state representation contains a 384-dimensional SentenceBERT embedding, word count, sentence count, three category indicators corresponding to

Mathematics, Logic, and Commonsense, and the remaining inference budget, resulting in a total state dimension of 390 features. RL_OVERVIEW The proposed system employs Disjoint LinUCB to dynamically select an appropriate combination of model size and reasoning strategy. Six inference actions are considered, ranging from computationally inexpensive direct inference using a 3B model to computationally intensive Tree-of-Thought reasoning using a 7B model. This enables the system to allocate computational resources according to the characteristics and expected difficulty of each query rather than applying a single inference strategy to all inputs. RL_OVERVIEW The reward formulation incorporates both correctness and inference efficiency:

$$R = 1.0(\text{Correct?}) - 0.3 \left(\frac{\text{Tokens Used}}{\text{Max Tokens}} \right) - 0.1 \left(\frac{\text{Time Taken}}{\text{Max Time}} \right)$$

This formulation encourages the router to prioritize correct answers while simultaneously penalizing excessive token consumption and inference time. Consequently, the objective of CascadeRoute is not simply to maximize accuracy, but to identify an effective accuracy–cost trade-off. The experimental evaluation demonstrated the effectiveness of this approach. On the combined evaluation benchmarks consisting of GSM8K, LogiQA, and CommonsenseQA, CascadeRoute achieved an accuracy of 81.4% while using an average of 1,421 tokens, compared with 82.0% accuracy and 1,959 tokens for the always-7B Tree-of-Thought strategy. This represents a reported 27.4% reduction in token consumption while maintaining accuracy close to the strongest fixed reasoning strategy. RL_OVERVIEW The learned routing behaviour further demonstrates the ability of the proposed system to differentiate between queries of varying computational requirements. Approximately 47.8% of queries were routed to inexpensive direct inference, 34.2% to Chain-of-Thought reasoning, and 17.7% to heavier Graph-of-Thought or Tree-of-Thought

reasoning. This distribution indicates that the system does not unnecessarily apply expensive reasoning to every query, but instead reserves higher computational effort for queries that benefit from it. The comparative evaluation of routing algorithms also showed the advantage of contextual learning. Among the evaluated approaches, LinUCB achieved the highest reported performance in the routing comparison, obtaining 69.3% accuracy, 794 average tokens, and a reward of 0.6020, outperforming random selection, rule-based routing, epsilon-greedy, Thompson sampling, UCB1, and DQN under that evaluation setting. A further component of CascadeRoute is the confidence-based escalation mechanism. The system evaluates a selected response using two temperature settings, $(T=0.2)$ and $(T=0.8)$. When both outputs agree, the response is accepted. When they conflict, the system progressively escalates the query through increasingly capable reasoning strategies. The project reports that this mechanism identified and corrected approximately 15–20% of exploratory routing mistakes, providing an additional reliability layer beyond the learned routing policy.

RL_OVERVIEW Overall, CascadeRoute demonstrates that reinforcement-learning-based contextual routing can provide a practical mechanism for adaptive LLM inference. Instead of treating model selection and reasoning strategy as fixed decisions, the proposed system learns to allocate computational resources according to query context. The results indicate that significant token savings can be achieved while retaining a high level of reasoning accuracy. Therefore, the primary contribution of this work is the integration of semantic query representation, contextual bandit learning, model selection, reasoning-strategy selection, budget awareness, and confidence-based escalation into a unified LLM inference routing framework. CascadeRoute provides a foundation for developing more efficient and adaptive LLM systems where computational resources can be dynamically allocated according to task requirements.

8.2 Key Contributions Of The Proposed Work

The major contributions of the project can be summarized as follows:

1. Context-Aware LLM Routing: CascadeRoute introduces a contextual routing mechanism that considers semantic and structural characteristics of the input query before selecting an inference strategy. **2. Joint Model and Reasoning Selection:** The system jointly considers model size and reasoning approach, selecting among six combinations ranging from lightweight 3B direct inference to 7B Tree-of-Thought reasoning. **3. Contextual Bandit-Based Decision Making:** Disjoint LinUCB is employed to continuously learn the relationship between query context, inference actions, reward, and computational cost. **4. Budget-Aware Inference:** Remaining computational budget is incorporated directly into the state representation, enabling the routing policy to account for resource constraints. **5. Cost-Aware Reward Function:** The reward function jointly considers correctness, token consumption, and inference time, thereby encouraging efficient yet accurate inference. **6. Confidence-Based Escalation:** A two-temperature confidence mechanism provides a safety layer that can detect disagreements and escalate difficult queries to more powerful reasoning strategies. **7. Reduced Token Consumption:** The proposed routing strategy achieved a reported 27.4% reduction in token consumption compared with the always-7B Tree-of-Thought strategy while maintaining comparable accuracy.

RL_OVERVIEW 8. Adaptive Computational Allocation: The learned routing distribution demonstrates that computationally inexpensive strategies can handle a substantial portion of queries, while expensive reasoning is reserved for more challenging cases.

8.3 Limitations Of The Present Work

Although the experimental results demonstrate the potential of CascadeRoute, several limitations remain.

First, the current evaluation is based on three benchmark datasets: GSM8K, LogiQA, and CommonsenseQA. These datasets provide coverage of mathematical reasoning, logical reasoning, and commonsense reasoning, but they do not represent the complete diversity of

real-world LLM workloads. Second, the current routing action space is limited to six predefined combinations of model size and reasoning strategy. A broader ecosystem containing additional model families, parameter sizes, prompting techniques, retrieval mechanisms, and specialized models could provide more routing possibilities. Third, the system relies on a predefined maximum-token and maximum-time normalization for the reward function. Different deployment environments may have substantially different latency, memory, and cost constraints. Consequently, the reward weights may need to be adapted according to the target deployment environment. Fourth, the confidence mechanism currently relies on agreement between responses generated using two temperature values. Although this mechanism provides a useful reliability signal, response agreement does not necessarily guarantee correctness. More sophisticated uncertainty estimation could therefore improve the reliability of the system. Finally, the present work primarily focuses on inference efficiency and does not explicitly incorporate other deployment-level factors such as GPU memory utilization, energy consumption, concurrent-user load, network latency, or monetary API pricing.

8.4 Future Work

The current CascadeRoute framework provides several opportunities for further development. Future work can focus on improving the routing policy, expanding the inference environment, strengthening reliability, and extending the system toward real-world deployment.

8.4.1 Expansion to Larger and More Diverse LLM Ecosystems

Future versions of CascadeRoute can incorporate a larger number of language models with different parameter sizes, architectures, and capabilities. Instead of restricting the action space to 3B and 7B models, the router could dynamically select from multiple small, medium, and large models. The action space could therefore be represented as:

$$A = \{(M_i, S_j) \mid M_i \in \mathcal{M}, S_j \in \mathcal{S}\}$$

where (M_i) represents an available model and (S_j) represents a reasoning strategy. This would transform CascadeRoute into a more general model-and-reasoning marketplace, where the router selects the most suitable inference configuration from a heterogeneous collection of LLMs.

8.4.2 Dynamic Budget-Aware Routing

Although the current system incorporates remaining budget into the contextual state, future work can make budget management more dynamic. The router could continuously estimate the expected computational cost of future queries and reserve resources accordingly. A future routing objective could be formulated as:

$$\max_{\pi} \mathbb{E} \left[\sum_{t=1}^T R_t \right]$$

subject to:

$$\sum_{t=1}^T C_t \leq B$$

where (C_t) represents the computational cost of the selected action and (B) represents the available inference budget. Such an approach would allow the router to make long-horizon resource allocation decisions rather than treating every query independently.

8.4.3 Advanced Uncertainty Estimation

The current confidence gate uses response agreement at different temperature settings. Future work can investigate more robust uncertainty estimation techniques. Potential approaches include:

- Multiple-sample response consistency
 - Entropy-based uncertainty
 - Token-level confidence estimation
 - Calibration models
 - Verifier or critic models
 - Self-evaluation mechanisms
 - Learned confidence predictors
- A learned confidence function could estimate:

$$P(\text{Correct} \mid s_t, a_t)$$

and use this probability to determine whether the current response should be accepted or escalated. This could reduce unnecessary escalation while improving the reliability of difficult queries.

8.4.4 Multi-Objective Reinforcement Learning

The current reward function combines correctness, token consumption, and inference time. Future work could explicitly formulate routing as a multi-objective optimization problem. For example:

$$R = w_1 A - w_2 C - w_3 T - w_4 E$$

where:

- (A) = accuracy,
 - (C) = token or monetary cost,
 - (T) = inference latency,
 - (E) = energy consumption,
 - (w_1, w_2, w_3, w_4) = configurable objective weights.
- This would allow CascadeRoute to adapt to different deployment scenarios. For example, a mobile or edge deployment may prioritize energy consumption, whereas a cloud API may prioritize monetary cost and latency.

8.4.5 Energy-Aware LLM Routing

An important extension would be the integration of energy consumption into the routing decision. Larger models and longer reasoning processes generally require greater computational resources. Future versions could collect hardware-level power measurements and incorporate them into the reward:

$$R_t = \alpha A_t - \beta C_t - \gamma T_t - \delta E_t$$

where E_t represents the energy consumed during inference. This would extend CascadeRoute from cost-aware routing toward energy-efficient and environmentally conscious LLM inference.

8.4.6 Continuous Online Learning

The current contextual-bandit formulation provides an appropriate foundation for online adaptation. Future work can investigate continuous learning from real-world user interactions. As new queries arrive, the router could update its parameters according to observed outcomes:

$$\theta_{t+1} = \theta_t + \Delta\theta_t$$

This would allow the system to adapt to changes in:

- Query distributions • Model capabilities • Model versions • Hardware conditions • User preferences • Computational budgets
- Such continuous adaptation could make the routing system more robust in long-running deployments.

8.4.7 Integration with Retrieval-Augmented Generation

Another promising direction is integrating Retrieval-Augmented Generation (RAG) into the routing action space. Some queries may require external knowledge rather than additional reasoning depth. The router could therefore choose among:

{Direct, CoT, GoT, ToT, RAG, RAG+Reasoning}

This would allow CascadeRoute to distinguish between queries that require deeper reasoning and queries where retrieval is more beneficial.

8.4.8 Real-Time and Multi-User Deployment

Future work can evaluate CascadeRoute in realistic multi-user environments. In such settings, routing decisions would need to consider not only the individual query but also the current system workload. The state representation could be extended with features such as:

- Current GPU utilization • Queue length • Number of concurrent requests • Available VRAM • Current latency • System-level energy consumption
- The resulting system could dynamically select models according to both query difficulty and current

infrastructure conditions.

8.4.9 Hardware and Edge Deployment

The current framework can also be extended to edge environments where computational resources are significantly more constrained than cloud infrastructure. A future edge-aware router could select models based on:

$$a_t^* = \arg \max_{a \in A} [\text{Expected Accuracy} - \lambda_1 \text{Latency} - \lambda_2 \text{Memory} - \lambda_3 \text{Energy}]$$

8.4.10 Extension from Contextual Bandits to Full Reinforcement Learning

The current system uses contextual bandits because individual queries are largely independent and the routing decision can be optimized based on immediate reward. Future work could investigate full reinforcement learning when sequential dependencies become important. For example, a future system could consider: $[s_t \rightarrow a_t \rightarrow r_t \rightarrow s_{t+1}]$ where the next state depends on previous routing decisions, accumulated budget, queue conditions, user interaction, and system load. Algorithms such as PPO, SAC, or actor-critic approaches could then be investigated for environments where routing decisions have long-term consequences.

8.5 Overall Future Direction

The long-term objective of CascadeRoute is to evolve from a query-level contextual bandit into a general-purpose intelligent inference orchestration framework. Such a system would dynamically coordinate models, reasoning strategies, retrieval systems, computational resources, and reliability mechanisms according to the requirements of each incoming task. The future architecture can be conceptualized as: Query $\rightarrow$ Context Understanding $\rightarrow$ Model Selection $\rightarrow$ Reasoning Selection $\rightarrow$ Inference $\rightarrow$ Confidence Estimation $\rightarrow$ Escalation/Routing $\rightarrow$ Reward Evaluation $\rightarrow$ Online Learning. This would enable LLM-based systems to move away from the conventional approach of using a single large model for every query and toward adaptive, resource-aware, and intelligent inference.

8.6 Final Conclusion

In conclusion, CascadeRoute demonstrates the feasibility of using contextual reinforcement-learning techniques to dynamically optimize LLM inference. By combining Sentence-BERT-based query representation, Disjoint LinUCB, multiple model sizes, multiple reasoning strategies, budget awareness, cost-sensitive rewards, and confidence-based escalation, the system provides a unified approach to balancing accuracy and computational efficiency. The experimental findings show that intelligent routing can substantially reduce inference cost while preserving high-quality reasoning performance. The reported 81.4% accuracy with 1,421 average tokens, compared with 82.0% accuracy with 1,959 tokens for the always-7B Tree-of-Thought strategy, demonstrates the central value of adaptive computation in LLM inference. RL_OVERVIEW The work therefore establishes a foundation for future research into cost-aware, adaptive, uncertainty-aware, and resource-constrained LLM systems. With extensions toward larger model ecosystems, dynamic budgets, improved uncertainty estimation, energy-aware routing, retrieval integration, and real-time multi-user deployment, CascadeRoute can evolve into a broader intelligent orchestration framework for efficient next-generation AI inference.

References

- [1] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance,” arXiv preprint arXiv:2305.05176, 2023. [2] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “RouteLLM: Learning to Route LLMs with Preference Data,” International Conference on Learning Representations (ICLR), 2025. [3] J. Dekoninck, M. Baader, and M. Vechev, “A Unified Approach to Routing and Cascading for LLMs,” in Proceedings of the 42nd International Conference on Machine Learning (ICML), PMLR, vol. 267, pp. 12987–13010, 2025. [4] N. Varshney and C. Baral, “Model Cascading: Towards Jointly Improving Efficiency and Accuracy of NLP Systems,” in Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 11007–11021, 2022. [5] L. Li, W. Chu, J. Langford, and R. E. Schapire, “A Contextual-Bandit Approach to Personalized News Article Recommendation,” in Proceedings of the 19th International Conference on World Wide Web (WWW), pp. 661–670, 2010. [6] W. Chu, L. Li, L. Reyzin, and R. E. Schapire, “Contextual Bandits with Linear Payoff Functions,” in Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR, vol. 15, pp. 208–214, 2011. [7] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022. [8] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” arXiv preprint arXiv:2305.10601, 2023. [9] M. Besta et al., “Graph of Thoughts: Solving Elaborate Problems with Large Language Models,” in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, no. 16, pp. 17682–17690, 2024. [10] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” International Conference on Learning Representations (ICLR), 2023.